



S82033: Optimizing KV Caches for LLM Inference: Dynamo KVBM, FlexKV, LMCache

March 18th 2026



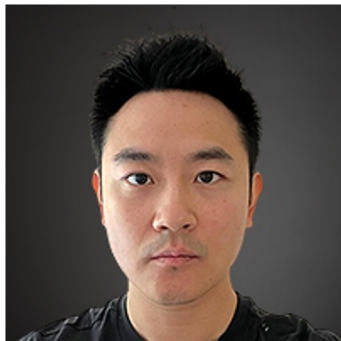
Speakers

Tutorial Host and Overview



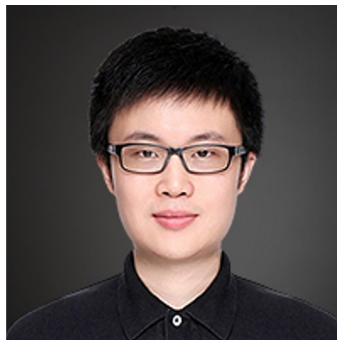
Vikram Sharma Mailthody
Sr Research Scientist,
NVIDIA

Dynamo KVBM



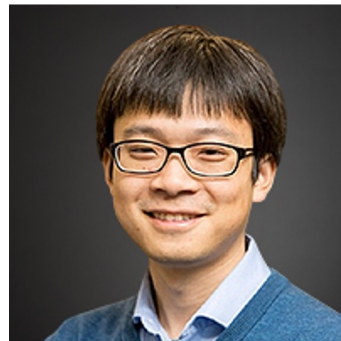
Ziqi Fan
Sr Software Engineer,
NVIDIA

Flex KV and Production experiences



Fan Ye
Dir. of Accelerated Computing,
Tencent

LMCache and Advanced KV uses



Junchen Jiang
CEO, Tensormesh Inc.
Professor, UChicago



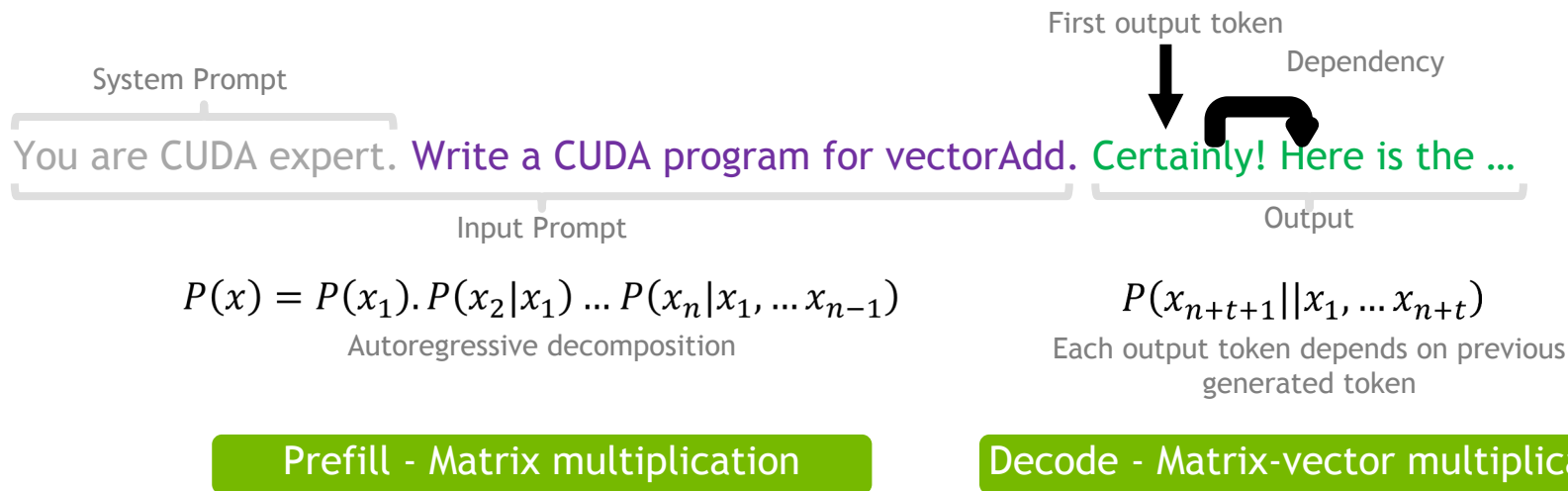
KV Caching 101

Vikram Sharma Mailthody

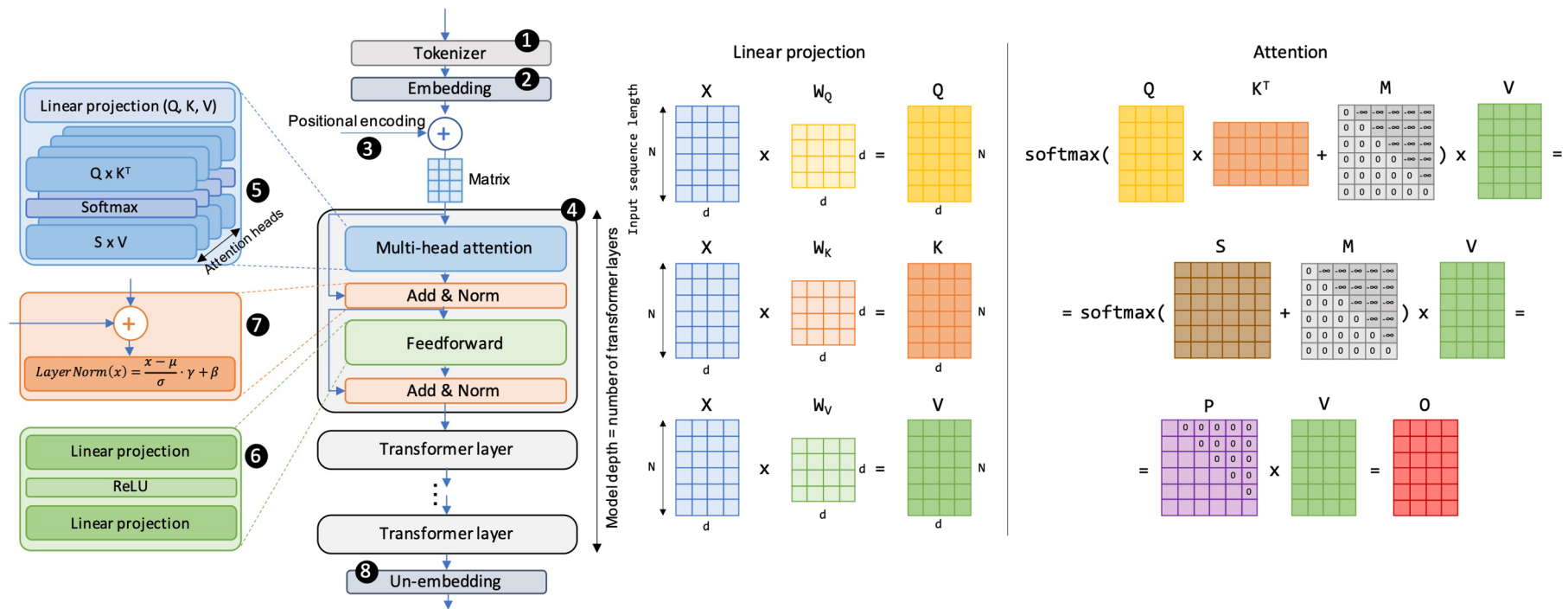
March 18th 2026



LLM Inference

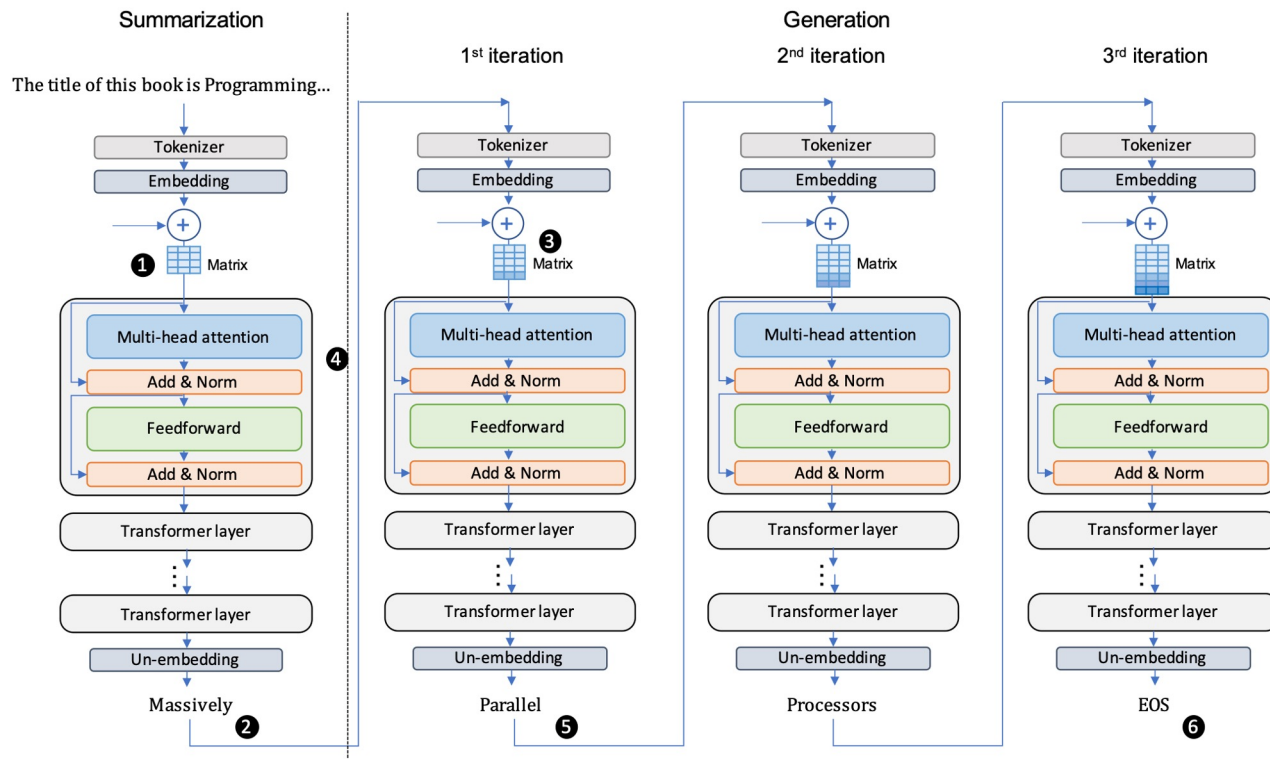


Transformer Computation



Credits: PMPP textbook v5.0 (yet to be released)

Next token prediction



Credits: PMPP textbook v5.0 (yet to be released)

Transformer Computation

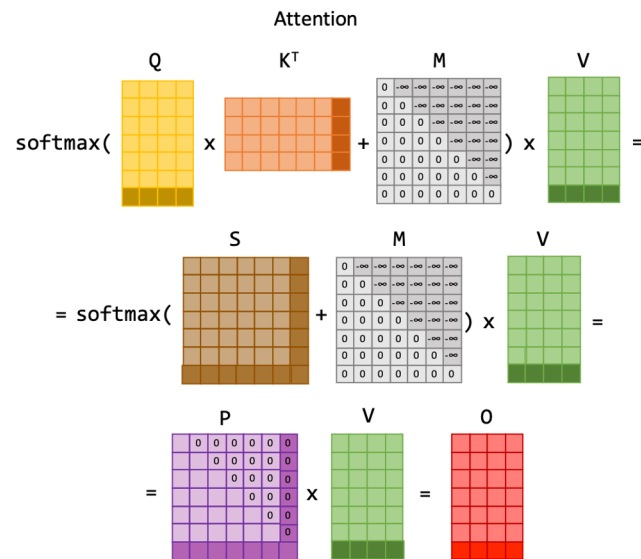
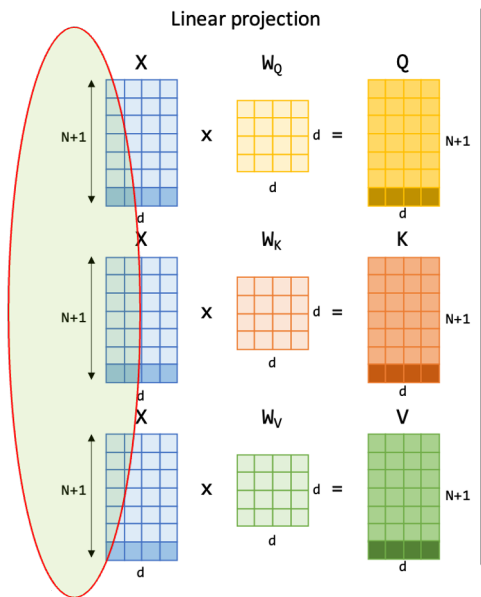
Five matrix multiplications are performed in each transformer layer

Seq len (N) can grow to millions → GEMM becomes expensive

Observation:

one additional row in the input matrix X from one iteration of the decoding phase to the next

Avoid repeating the computation exploiting GEMM property



Transformer Computation

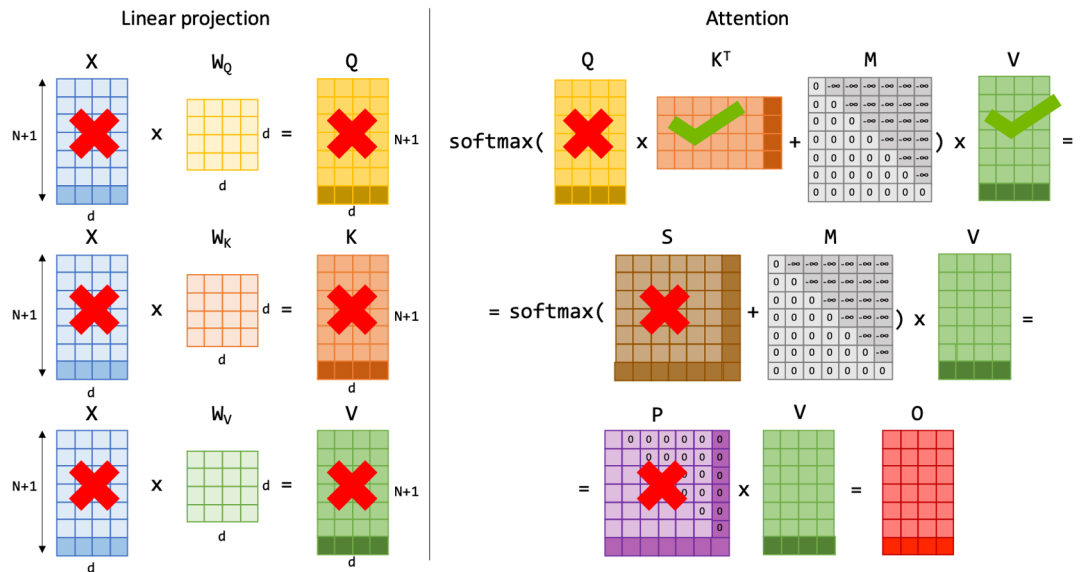
Five matrix multiplications are performed in each transformer layer

Seq len (N) can grow to millions $\rightarrow$ GEMM becomes expensive

Observation:

one additional row in the input matrix X from one iteration of the decoding phase to the next

Avoid repeating the computation exploiting GEMM property

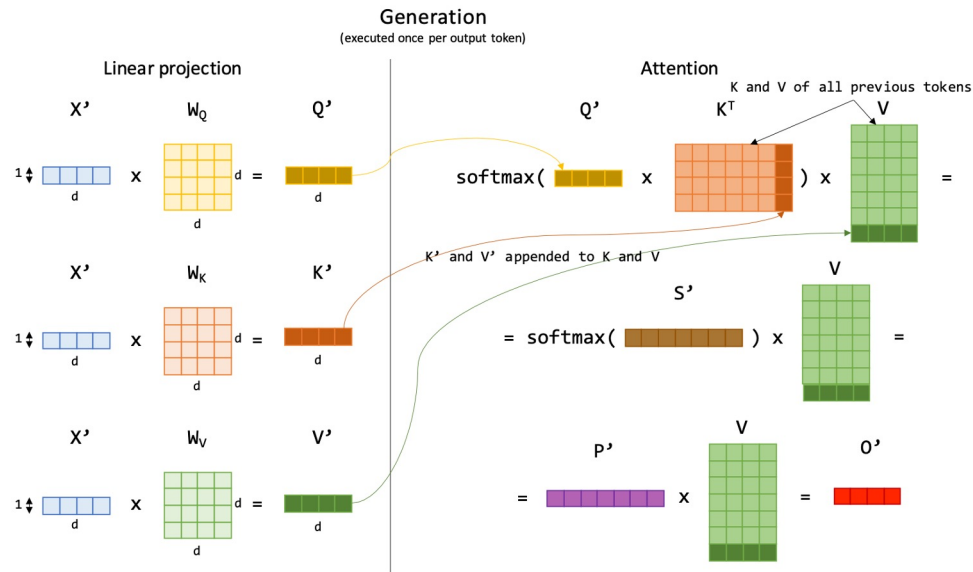


KV Caching

Reusing the computations saves a lot of compute at the cost of memory

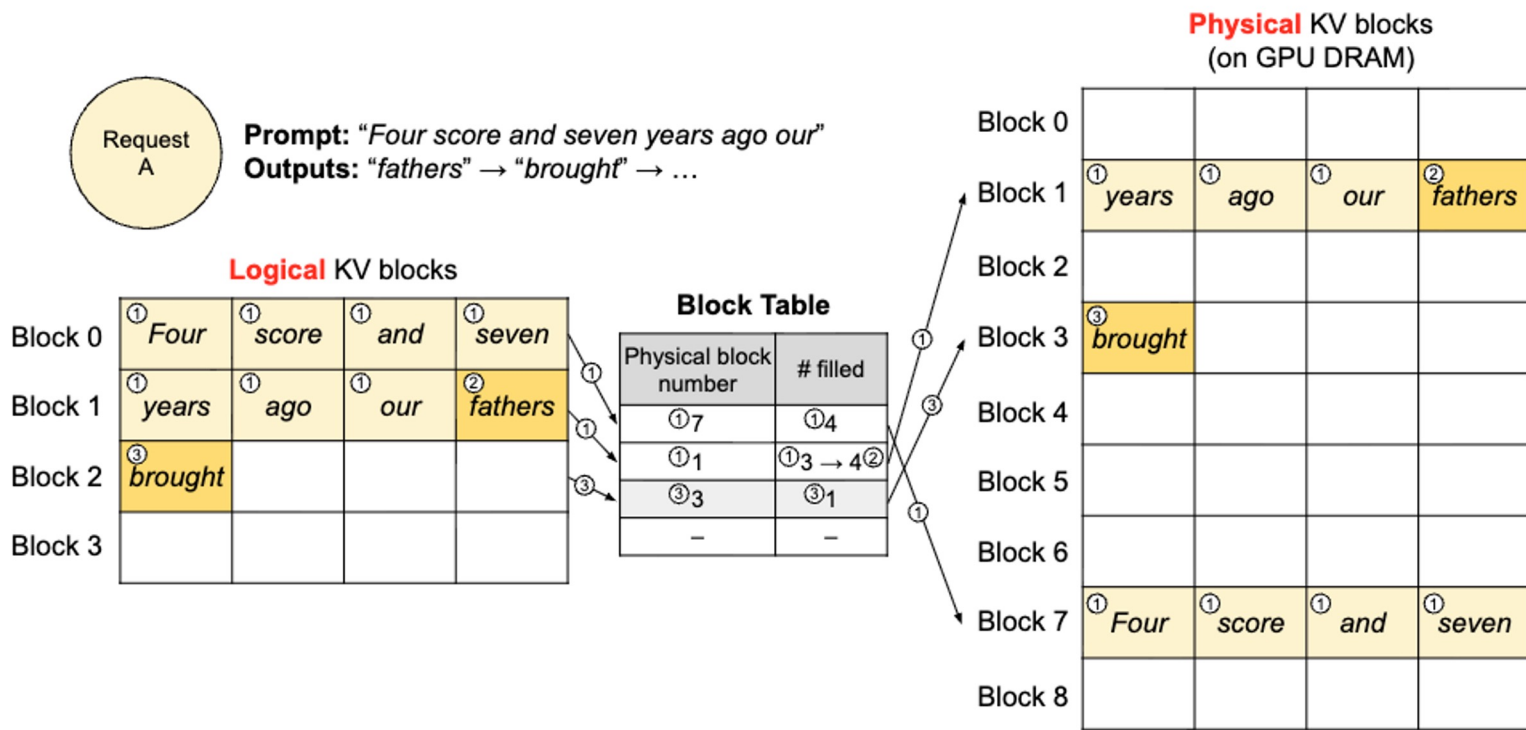
But memory needs to be allocated and managed efficiently

- PagedAttention (vLLM)
- RadixAttention (SGLang) across requests



PagedAttention

Addressing memory fragmentation issue



Production systems

More complex deployments

Multiple requests with
varying ISL/ OSLs

Agentic tasks and Multi
turn conversations

Multiple models to serve

Need to SLA both TTFT
and TPOT

Optimize to maximize KV
cache hit rate

Fault tolerance

AI Factory TCO at scale

Optimize for power and
goodput

Compressions and
blending techniques



LLM Deployments in 2026

Scale Never Seen Before



Input Length

1M+ with RAG, Video understanding, Code...



Output Length

Reasoning, Agentic Workflows...



Model Size

O(100B) – O(1T) parameters



Efficiency

More tokens = More revenue



Deployment Constraints

Power, HW availability (memory!), cooling...

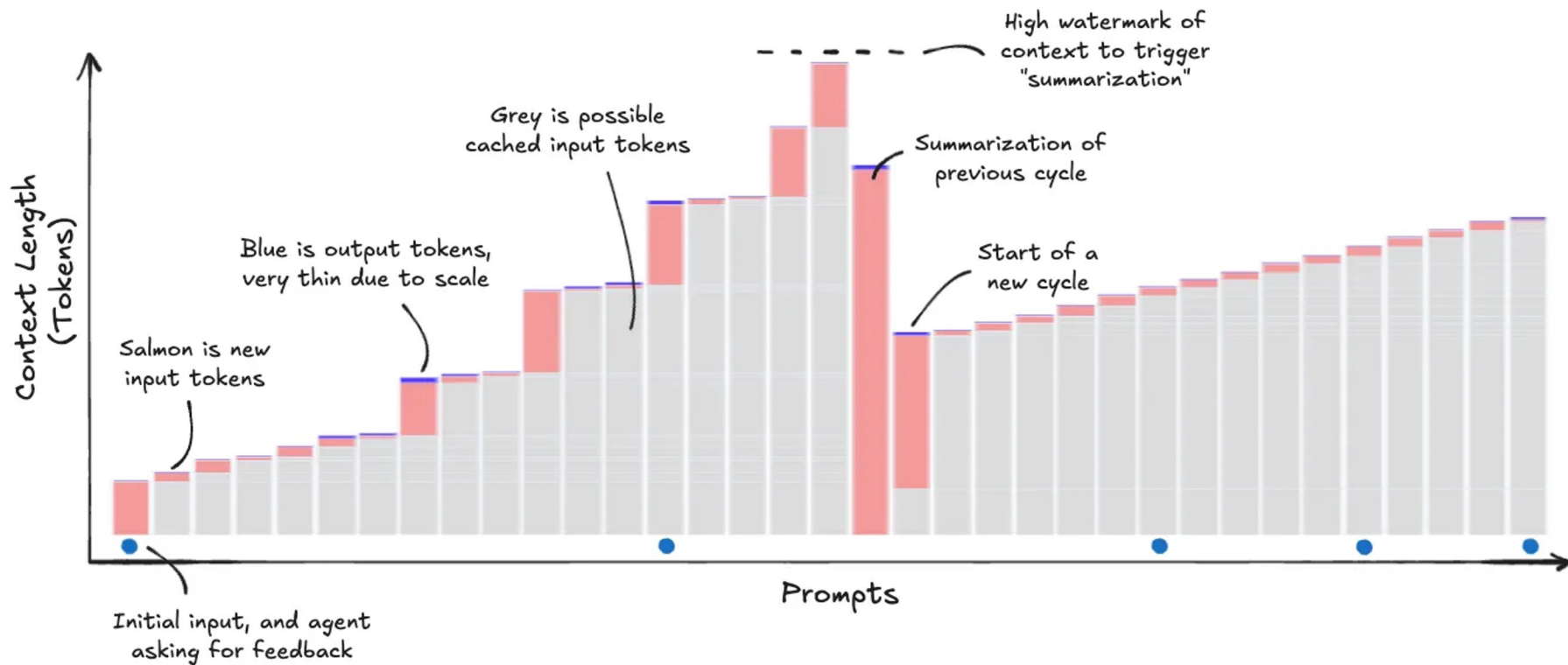
Agentic AI Redefines Inference

Multi-turn, reasoning-based AI introduces new inference infrastructure demands

- AI shifts from chatbots to multi-step workflows
- Agents require persistent inference context
- Context grows with longer sequences and sessions
- KV cache becomes long-lived AI memory
- Performance depends on storing and reusing context



Agentic AI redefines Inference



Courtesy Callan Fox

Inference Context is the New Bottleneck

Storage must be rearchitected

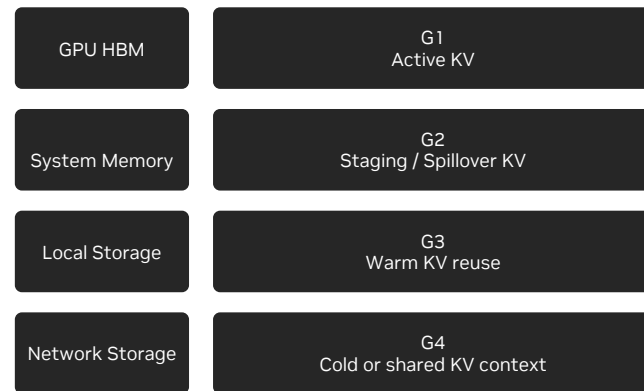
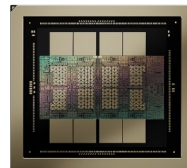
Inference context is large, dynamic, and recomputable

Context must be shared across GPUs and nodes

Local memory is inherently limited

Scaling traditional storage increases cost and power

Efficient context scaling demands a new architecture



Context Memory Hierarchy



NVIDIA Dynamo KVBM

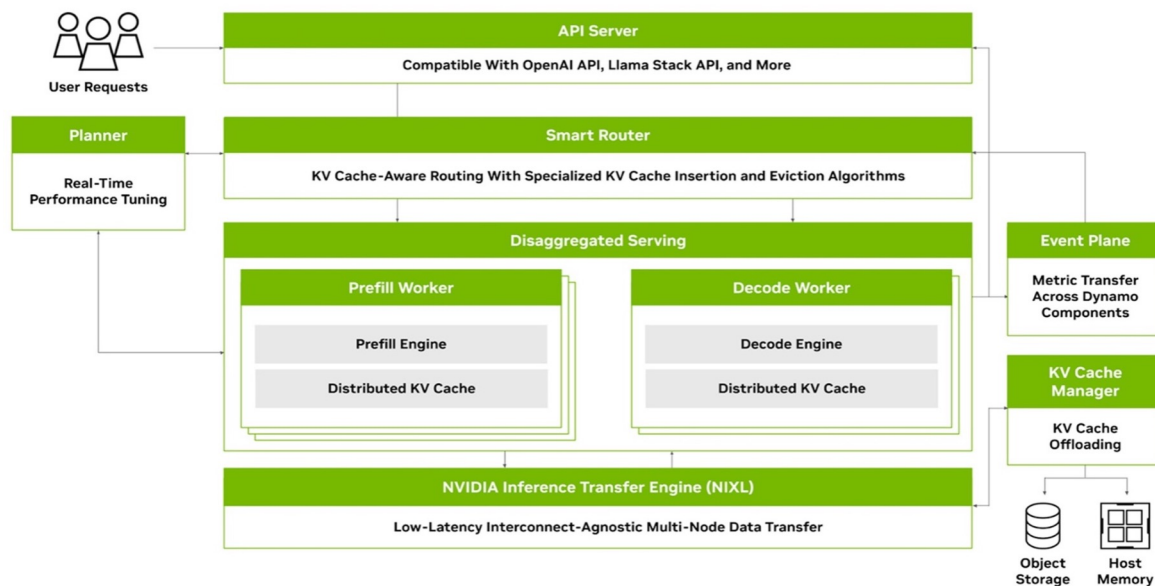
Ziqi Fan

March 18th 2026



NVIDIA Dynamo: Distributed Inference At Scale

Highly efficient production grade open source framework with modular design

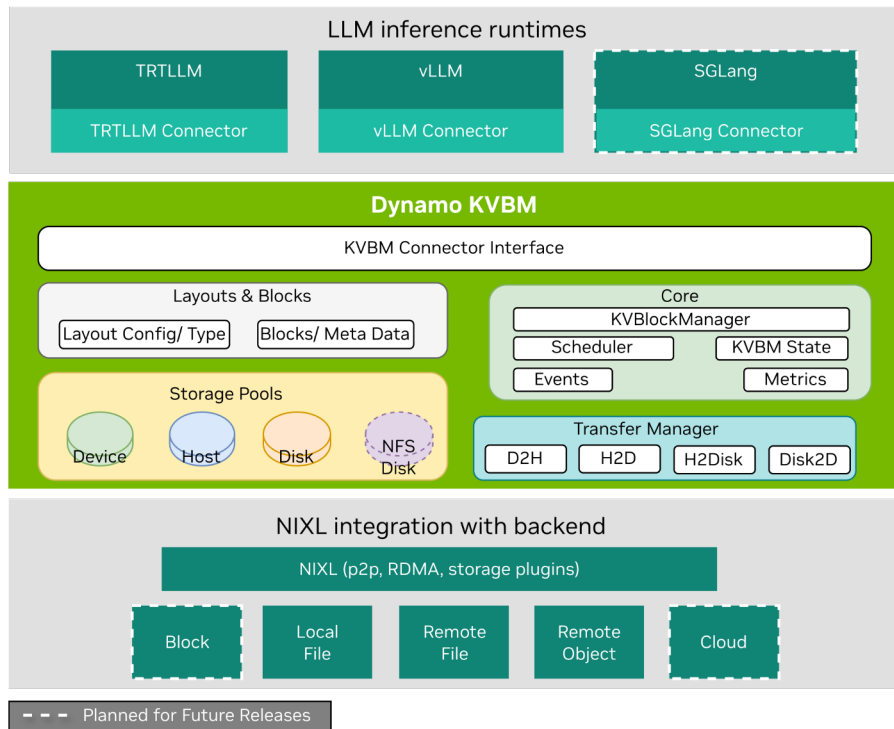


10+ notable
companies in
production
20K+ GPUs

204 contributors
(92 external)

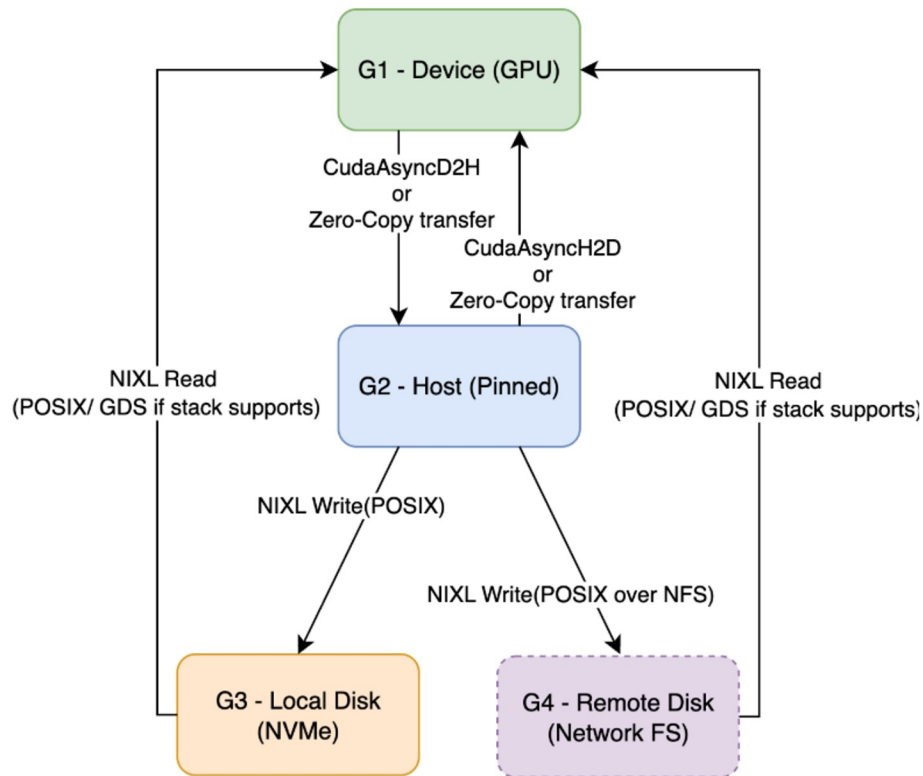
~90 commits per
week

KVBM Architecture and components



https://docs.nvidia.com/dynamo/latest/kvbm/kvbm_architecture.html

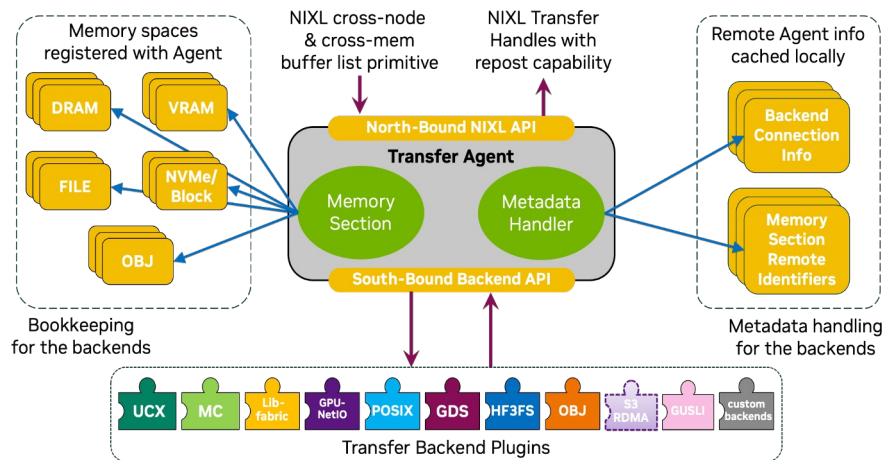
KV Transfer Manager



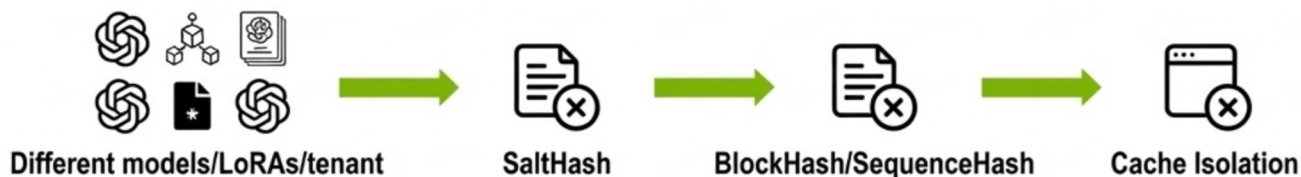
NIXL Storage Interface for Backend Integration

NIXL provides abstractions over various types of memory (e.g., CPU and GPU) and storage

- KVBM utilizes NIXL for storing and retrieving KV Blocks from storage.
- NIXL provides high-performance unified API for heterogeneous architectures
- Asynchronous, non-blocking, non-contiguous and Zero Copy.

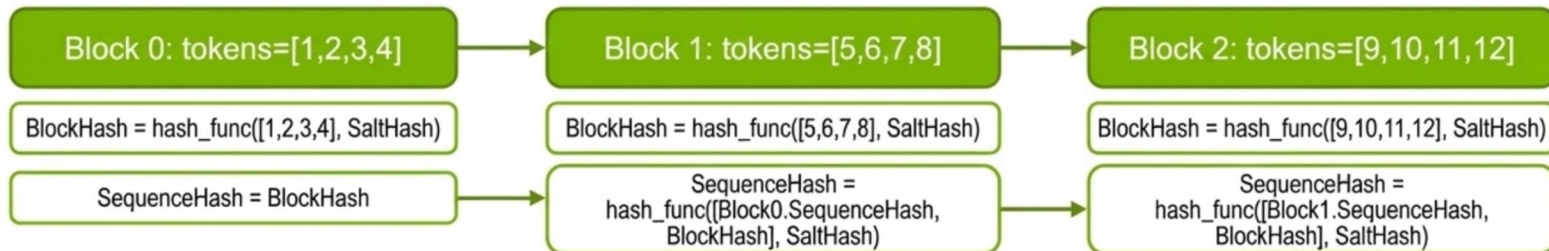


Block Hash | Sequence Hash | Salt Hash



The seed ensures different contexts produce different hashes even with the same input data.

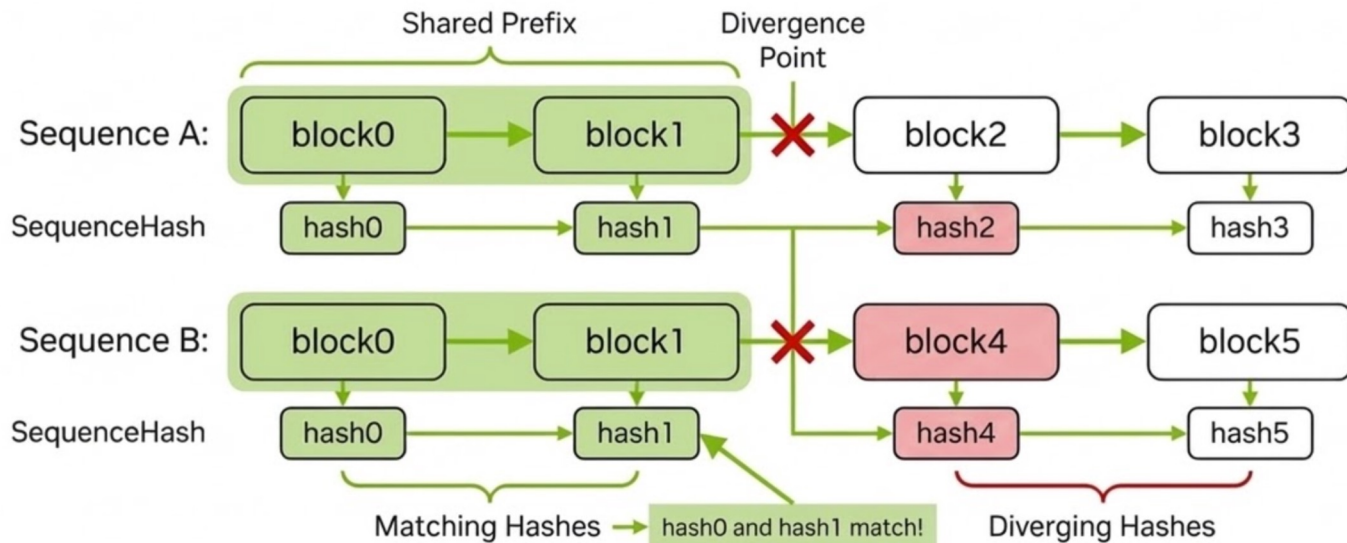
Data structure example



Prefix Cache Matching

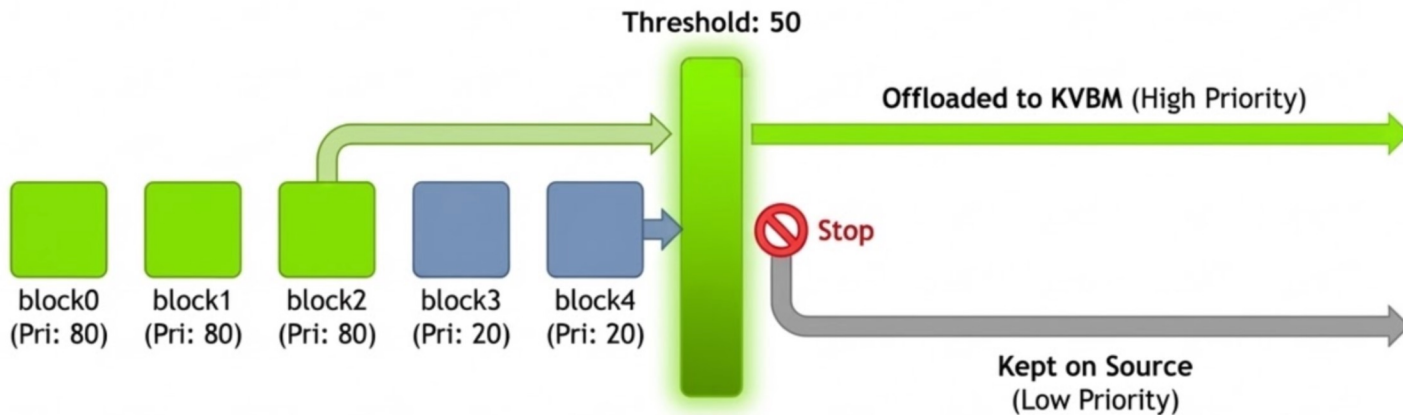


Key Insight: If two sequences share a prefix, their SequenceHashes will match up to the point of divergence.



Selective Offloading with User Hints

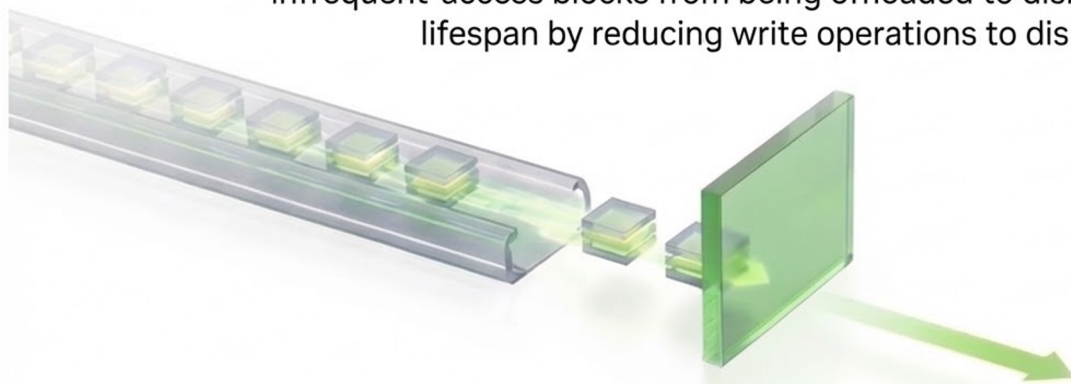
Leverage user hints to selectively offload **high-priority** KV blocks, minimizing transfer overhead.



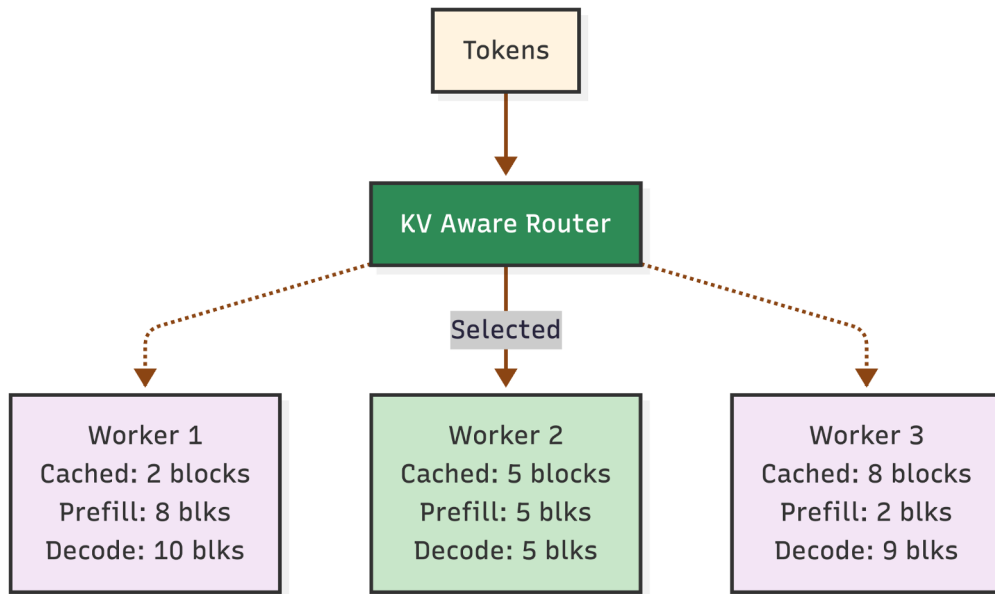
Result: Only blocks with Priority ≥ 50 are offloaded.
Offload [block0, block1, block2]. Stop at block3.

Offload Filter

The disk offload filter uses a **frequency-based filtering mechanism** to prevent infrequent-access blocks from being offloaded to disk. This protects SSD lifespan by reducing write operations to disk storage.



Dynamo KV Aware Router



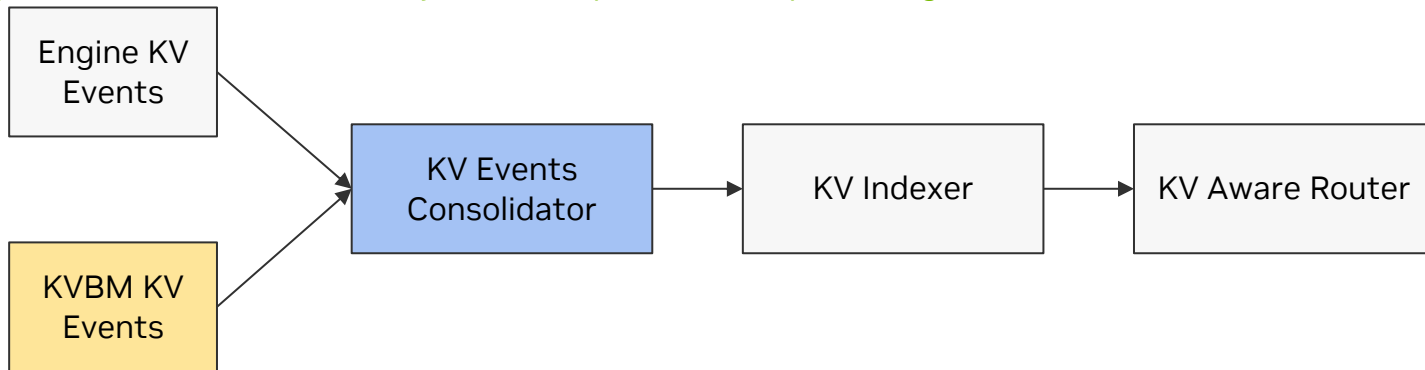
prefill load = input blocks - cached blocks = $10 - 5 = 5$

decode load = active blocks + new blocks = $5 + 10 = 15$

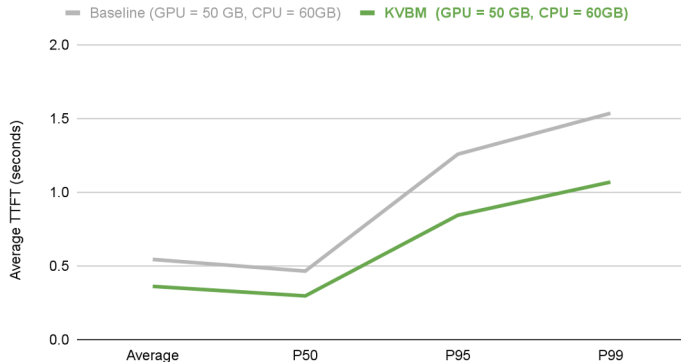
cost = γ * prefill load + decode load = $1 * 5 + 15 = 20$

Integration of KVBM with KV Aware Routing

Seeing E2E benefits with several dynamo components composed together

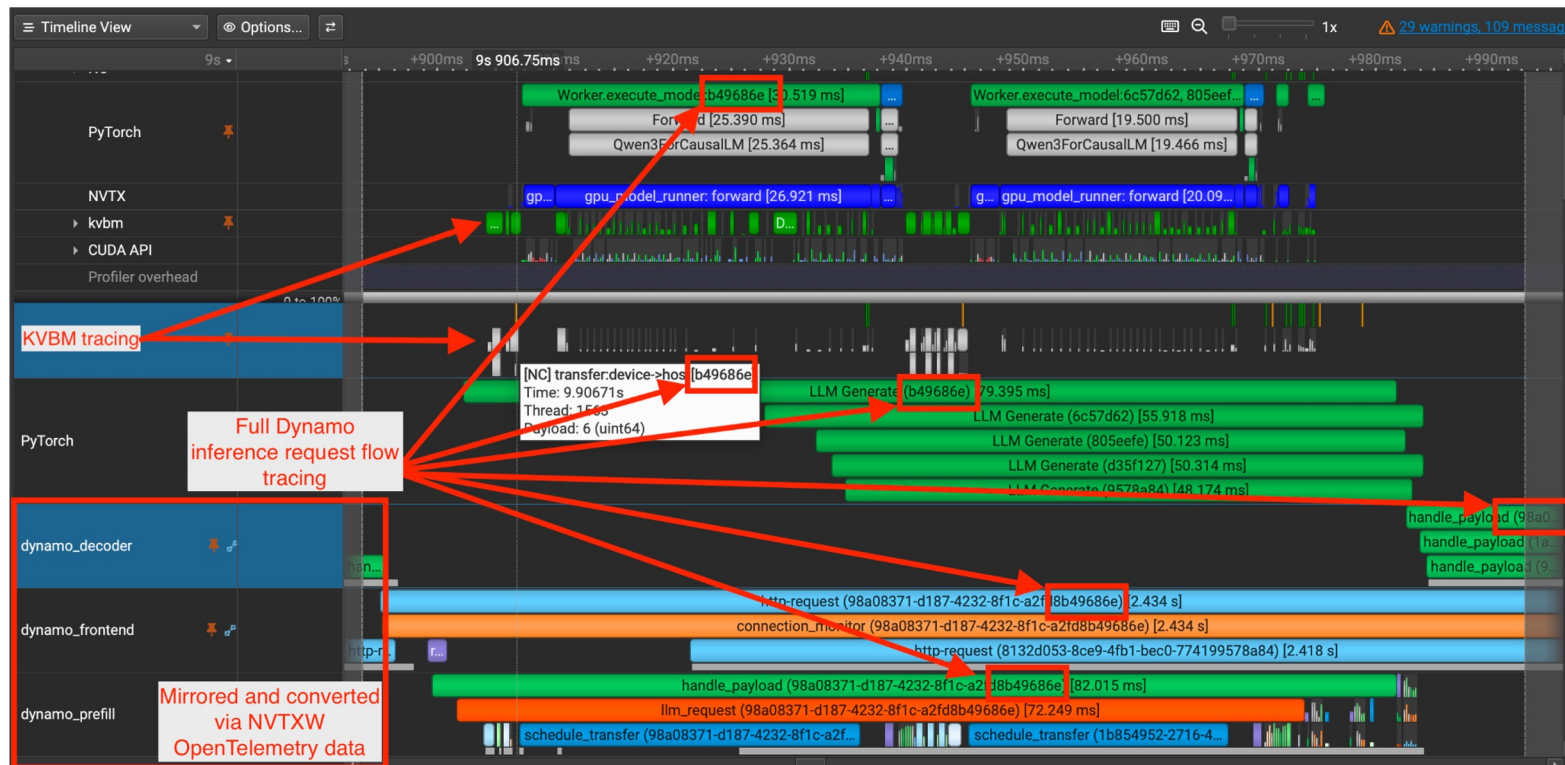


Qwen3-32B | vLLM Agg | TP=1 | Mooncake trace benchmark



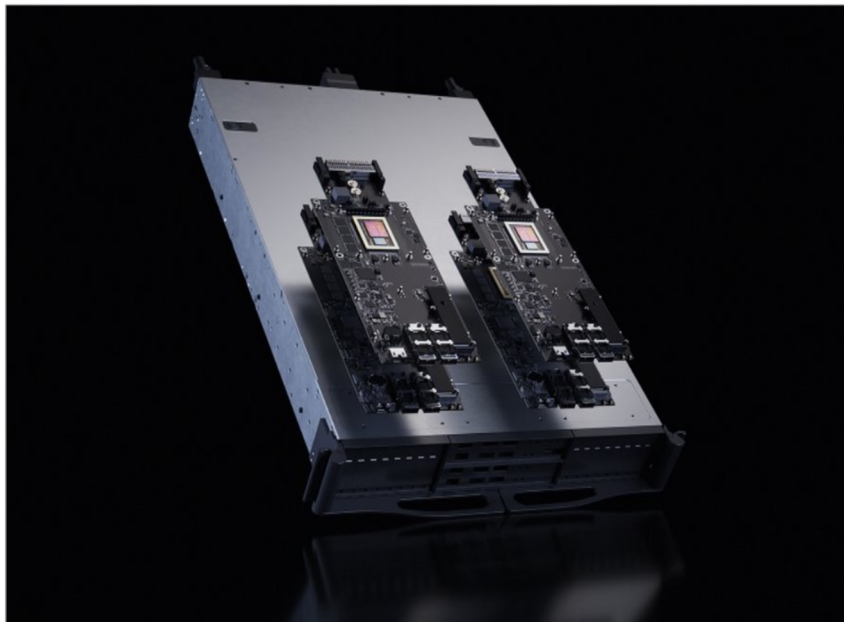
- **33.6% average improvement in TTFT** by enabling KV events
- Also see 7% average improvement in Output Token Throughput per user.

Perf Analysis with Nsight Systems



<https://developer.nvidia.com/nsight-systems>

NVIDIA Inference Context Memory Storage Platform



NVIDIA Inference Context Memory Storage Platform

- AI-native, pod-level context storage tier
- Shared across GPUs for efficient context reuse
- Complements GPU memory, local, and networked storage
- Improves performance and scale, reducing TCO:
 - 5x higher throughput (tokens-per-second) than baseline without additional capacity
 - 5x better power efficiency than traditional storage
- Accelerated by BlueField-4, Spectrum-X Ethernet, DOCA, NIXL, and Dynamo
- Designed by NVIDIA and built by storage industry leaders

AIC

CLLOUDIAN

ddn

DELL Technologies

HPE

Hitachi Vantara

IBM

NUTANIX

PURE STORAGE

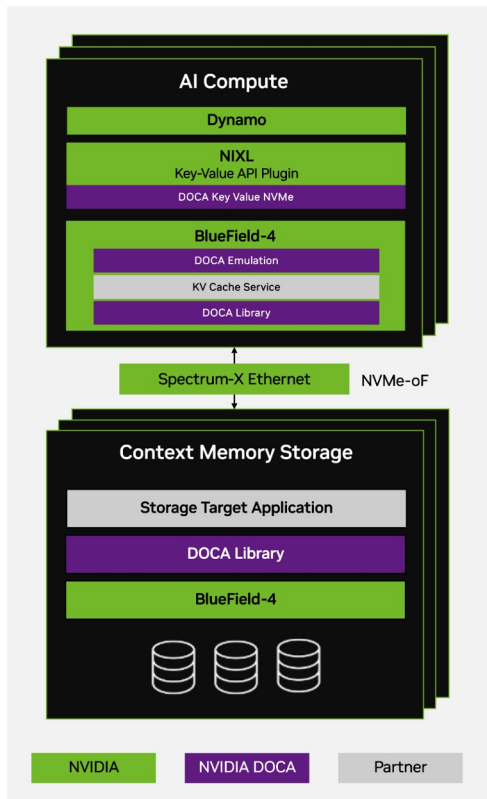
SUPERMICRO

VAST

WEKA

<https://developer.nvidia.com/blog/introducing-nvidia-bluefield-4-powered-inference-context-memory-storage-platform-for-the-next-frontier-of-ai/>

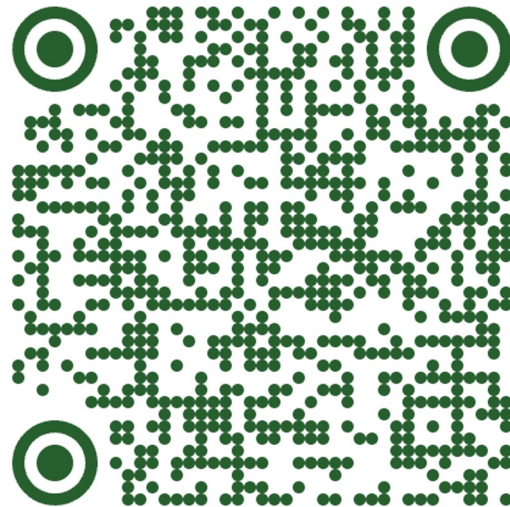
Dynamo KVBM + ICMS



- Efficient KV cache management and sharing
- Runs on BlueField-4 at compute and context memory nodes
- POD level scalable – application remains stateless
- Key-value APIs expose context directly to AI compute
- Secured and isolated KV cache access
- Hardware-accelerated integrity and encryption
- Developer-ready, extensible framework

Get Started With KVBM

- User guide [\[link\]](#)
 - `pip install kvbm`
- Contribute:
 - Rust Core [\[link\]](#)
 - Python Bindings [\[link\]](#)





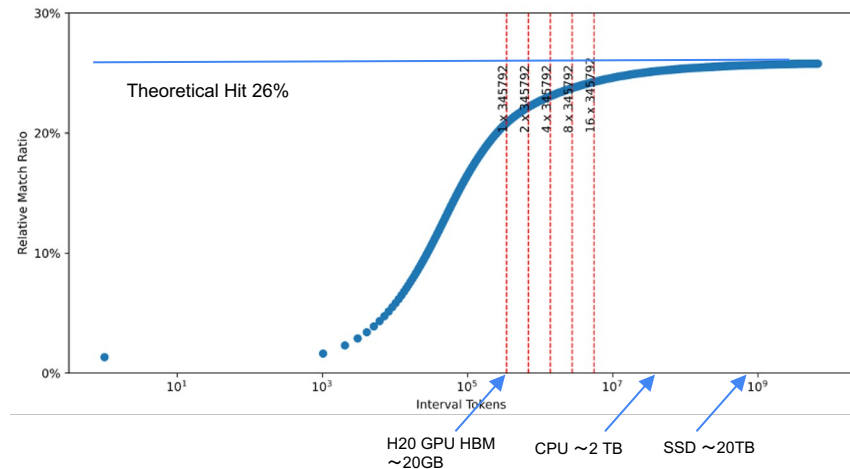
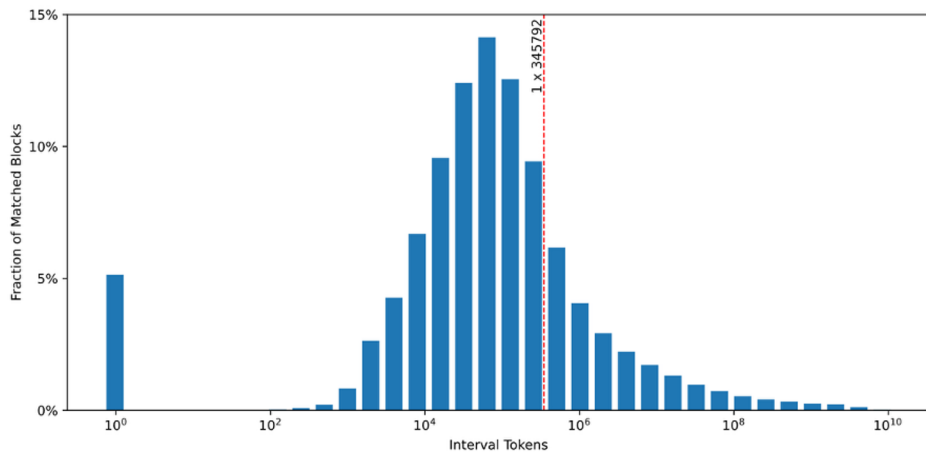
FlexKV and KV Cache in production

Fan Ye

Director of Accelerated Computing, Tencent

Production problems: extra kvcache hit potentials

Code Assistant: Observe potential gains by kvcache offloading



Interval tokens of a token is use to evaluate the reuse potential of this token.

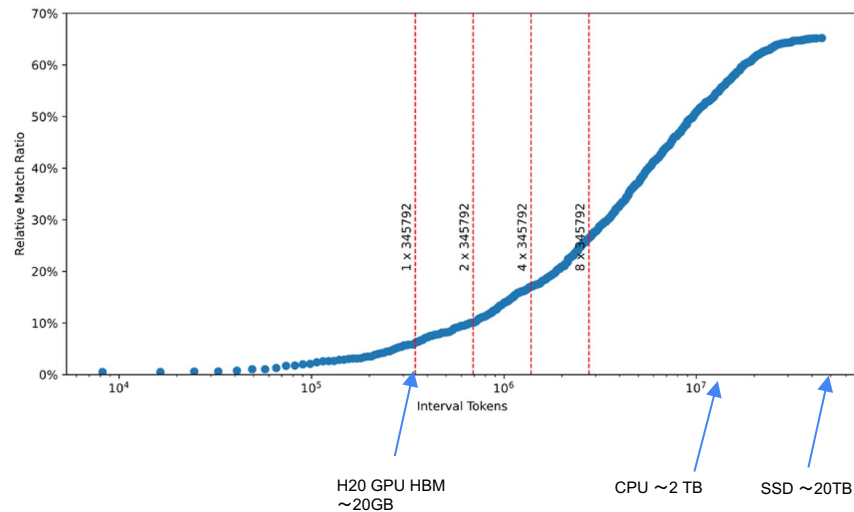
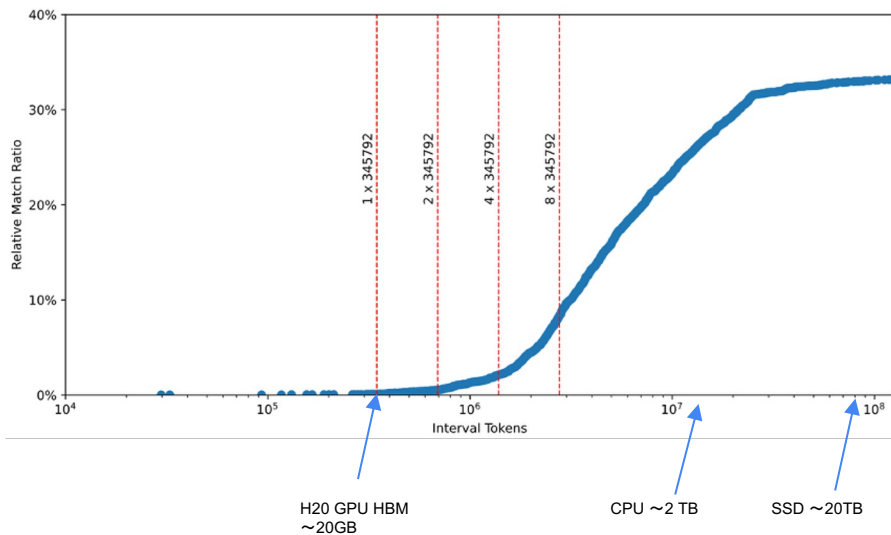
GPU HBM is limited, especially when a large portion of it is used for parameters and intermediate results

The GPU capacity is about 345K tokens, and a lot of tokens has longer interval tokens than that, which we can expect extra kvcache hit if we have a kvcache offloading implementation.

Production problems: extra kvcache hit potentials



Mooncake trace: Even larger kvcache extra kvcache hit potentials



Interval tokens of tokens in mooncake trace is larger, which means better extra hit rate if we have kvcache offloading

Some production dataset (Code Assistant) has low extra hit rate, while Mooncake trace has better hit rate. Therefore, we need an offloading implementation that has performance gains in all scenario.

Production problems: extra kvcache hit potentials

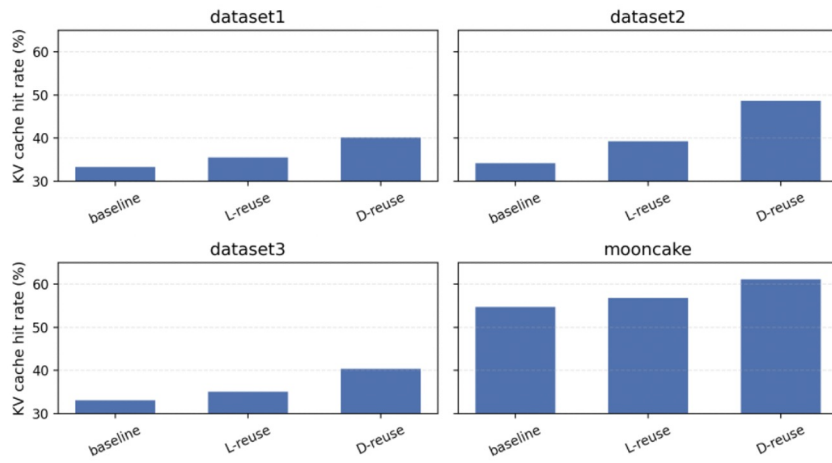
Distributed kvcache reuse brings about significant better kvcache hit rate

Distributed kvcache reuse is a necessity as the number of users increases.

We use three datasets (dataset1,2,3) and mooncake trace to see how much extra cache hit rate we can expect from distributed kvcache reuse, in 8-node inference scenario.

L-reuse stands for local kvcache hit rate, while D-reuse denotes distributed kvcache reuse

We can expect 4%~10% extra hit rate when distributed kvcache



Production problems: extra kvcache hit potentials



NitroFS: Tencent High-performance Cloud Storage

NitroFS: Tencent Cloud high-performance parallel file system designed for LLM training and infer

- Ultra-fast KV Cache I/O reaching near-local SSD speeds
- Virtually unlimited PB-level storage capacity, enabling KV Cache retention for several months
- Inherent cross-node KV Cache sharing that bypasses the complexities of traditional cross-node communication.
- multi-node read/write speeds scaling up to TB/s

NitroFS can serve as a backend for distributed kvcache.

FlexKV overview



Background and problems

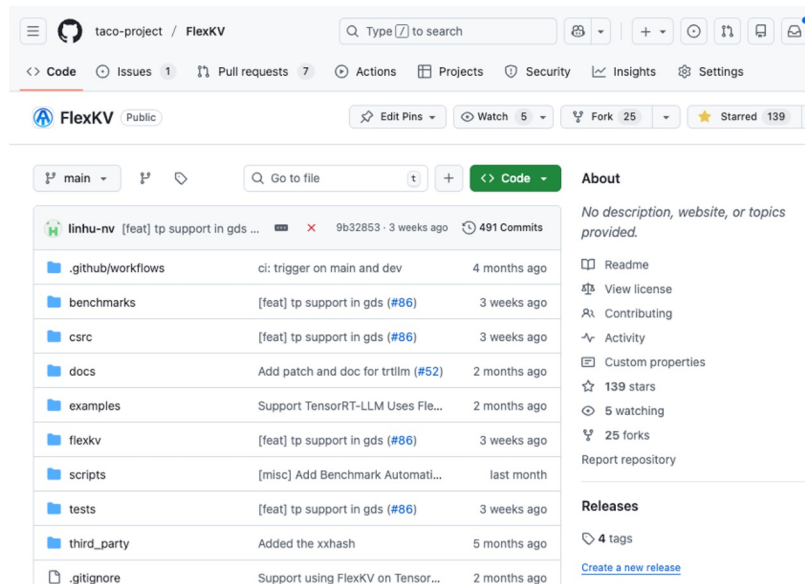
- Kvcache reuse can bring great performance improvements
- HBM is the key bottleneck in kvcache managements
- Cache hit rate may be unstable, so we need to minimum extra overhead
- Distributed serving is getting increasingly important

Overview

- Kvcache management system and Disaggregated memory system
- High-performance
- Easy-to-use APIs, inference engine integrations, transparent underlying designs

Key conception

- Offloading and reuse
- Tiered storage
- Indexing and transfer
- Distributed reuse



Key Features



Functionality

- support distributed kvcache lookup and reuse for multiple inference instances
- one flexkv instance can serve multiple DP client
- support cross-node TP groups
- multiple storage backend: ssd, GDS, distributed file system and RDMA-connected remote memory
- layerwise transfer
- namespace isolation
- GPU direct NVMeoF transfer (WIP)

Performance

- Low hit rate tolerance: we can expect e2e performance gains when the extra hit rate is above 1.5%
- 3.1% throughput improvements under 5% hit rate
- 21% throughput improvements under 20% hit rate
- 1.8x throughput improvements under 50% hit rate
- 6.15x throughput improvements under 100% hit rate

Framework

Dynamo/vllm/trtllm/SGLang

Architecture deepdive

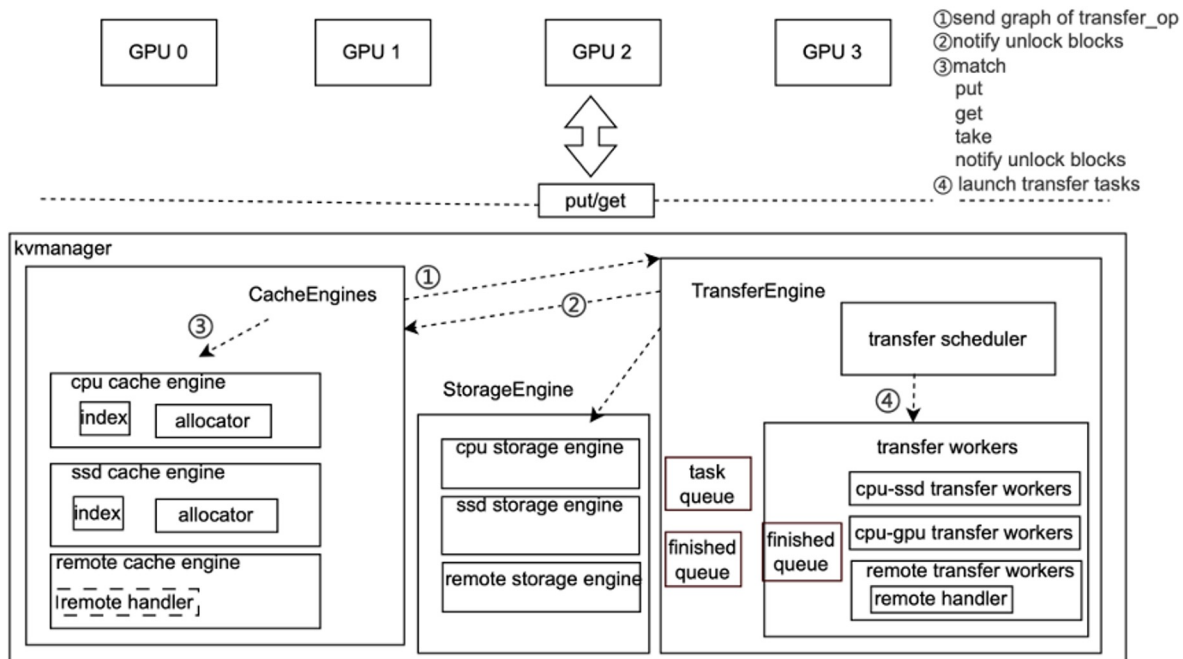
Three main modules:

- cache engine
- transfer engine
- storage engine

Fully pipelined between cacheEngine and transferEngine

Detached submodule between multiple hierarchies: more flexible and easy-to-maintain

One-serve-all: one flexKV instance can serve multiple gpus



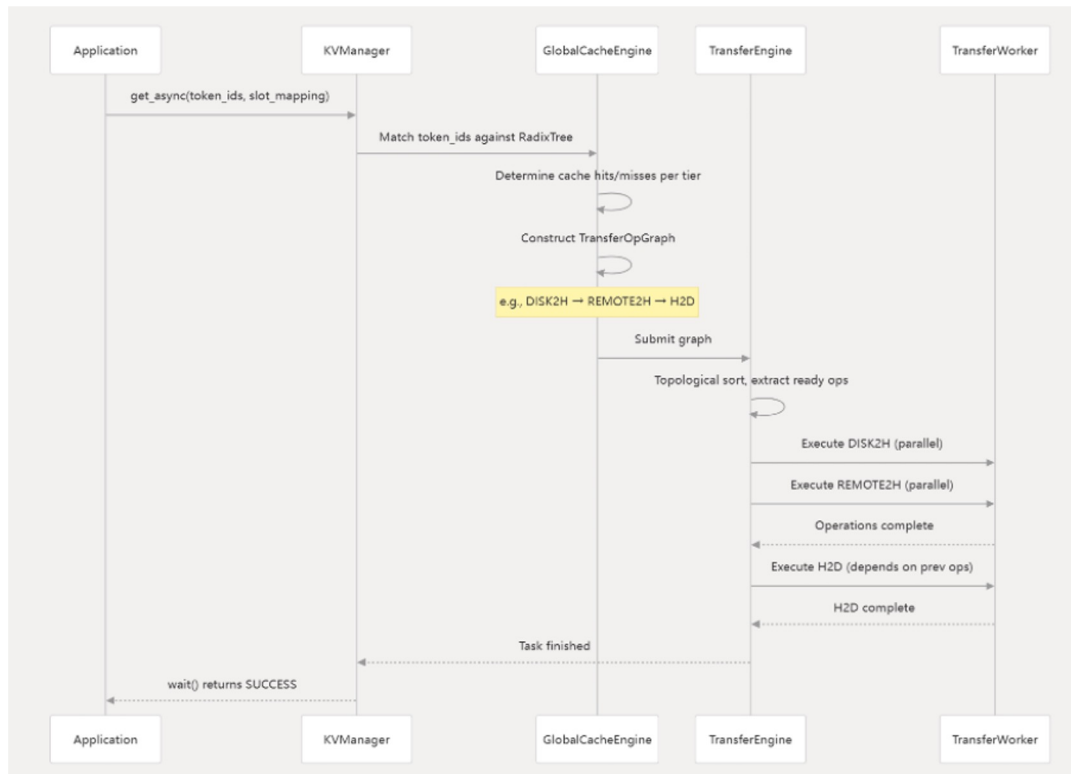
Data Flow: GET and PUT Operations

GET Operation Flow

- Match token sequence against tier-specific RadixTrees
- Construct transfer graph
- Execute transfers in dependency order
- Signal completion and unlock resources

PUT Operation Flow

- Match to determine if data already exists
- Construct transfer graph
- Execute eviction if caches are full
- Signal completion



Architecture and design highlights



Cache engine

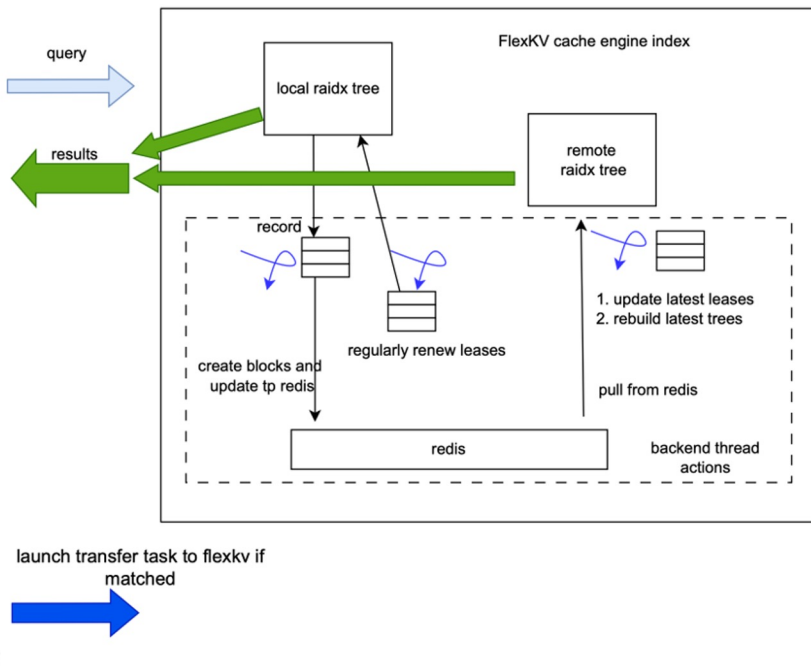
Close to LLM inference scheduler

- Low match latency

High performance radix tree

- c++ accelerated
- Small tree height due to tree node compaction
- Less memory access compared with hash-based method
- LocalRadixTree and DistributedRadixTree
- to support distributed kv reuse

Inference engine



Architecture and design highlights

Transfer engine



Separate Processes for different transfer workers

- Fully pipelined
- No performance decrease to inference engine

Multiple storage backend supported: GDS, distributed file system, rdma-based peer cpu access

Highly optimized transfer worker

- IOUring and Async I/O, achieving SoL SSD bandwidth, about 6GB/s on PCIe gen4, and scale linearly with the number of ssds
- Flexible datalayout: layer-first, block-first
- Highly optimized kernels for CPU-GPU transfer

Architecture and design highlights

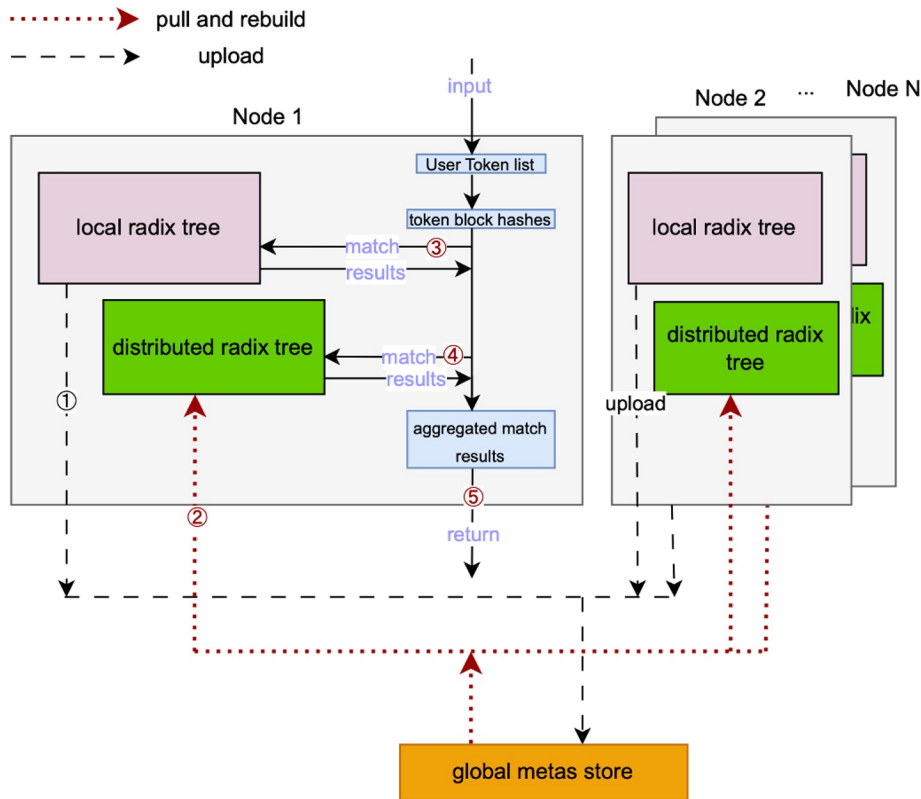
Distributed kv reuse

The most popular method for kvcache meta info sharing is centralized meta server, and all inference node will upload local meta and lookup peer meta in it.

Centralized method suffers from long lookup latency (through network) and heavy workloads for the centralized server.

We design and implement a “upload & rebuild” pattern

- upload meta & pull meta & rebuild
- backend threads do heavy works
- No network latency, so extremely low lookup latency (about 20us)
- “Lease” to assure final consistency

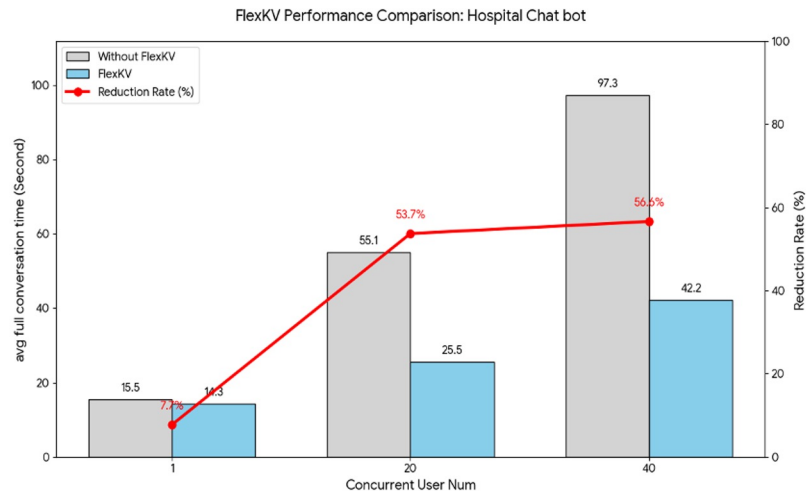
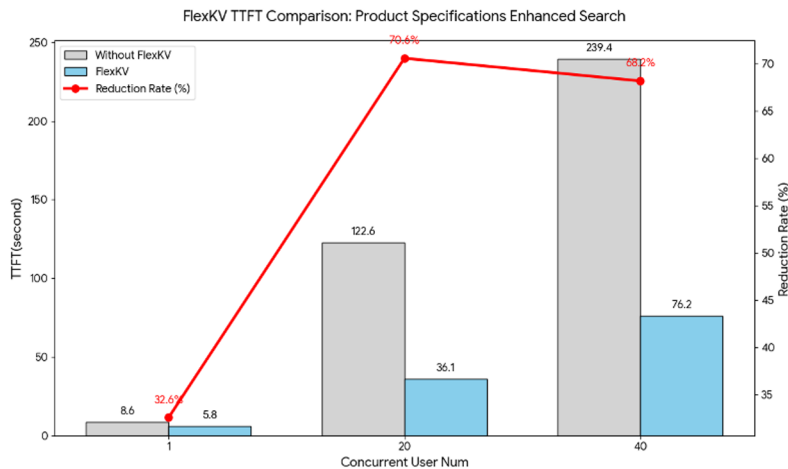


User case and Production Deployment

Phison

Phison

- Product description enhanced search: Use FlexKV to pre-cache product documentation KV cache so user queries can directly reuse it, reducing TTFT by 70%.
- Medical Q&A assistant: Use FlexKV to pre-cache KV cache for multiple medical knowledge bases so user questions can be retrieved instantly, reducing conversation latency by 57%.

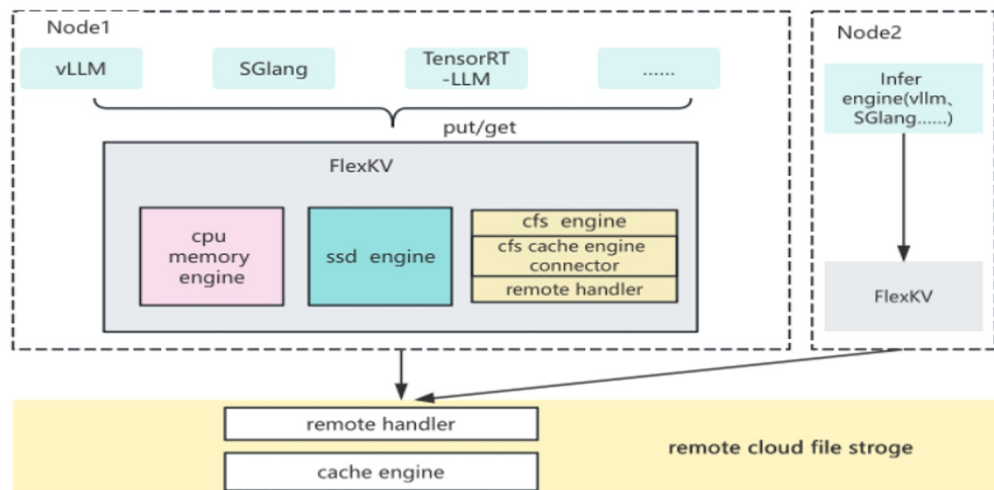
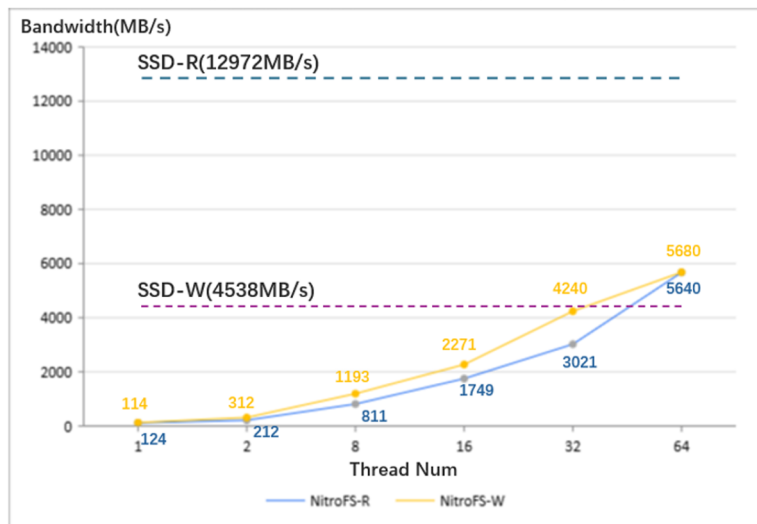


User case and Production Deployment

Integration with NitroFS



FlexKV NitroFS & SSD 32KB IO Bandwidth



User case and Production Deployment

Performance improvements under different hit rate

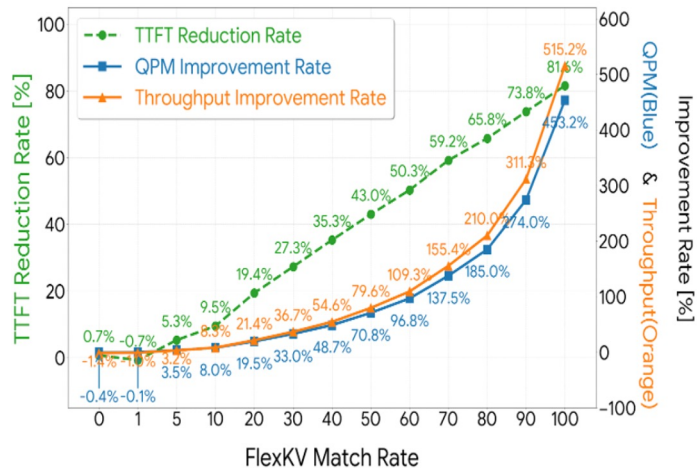


Code Assistant Case:

- The short-interval nature of code assistance cause the majority of the information to remain resident in the GPU HBM, and limits the "available" hit space for external KV caches. But we have successful overhead control in FlexKV.
- Actual FlexKV KVCache Hit Rates(4-5%), FlexKV still achieves a 3.4% reduction(improvement) in TTFT, ensuring a net-positive gain even under low hit-rate conditions.

Mooncake Trace:

- Its reuse intervals for multi-turn conversation data are typically long, requiring FlexKV to capture 25%–40% of the repetitive KV cache.
- FlexKV typically achieves massive performance improvements on such datasets.
(Setup: 8x H20, TP=8 (TP+EP), DeepSeek-R1-W4AF8, Synthetic 2k input/20 output tokens)



Resource Savings :

- Code Assistant Case : 16.67 GPU-Hours saved per 1M requests.
- Multi-turn Conversation : 95.42 GPU-Hours saved per 1M requests.

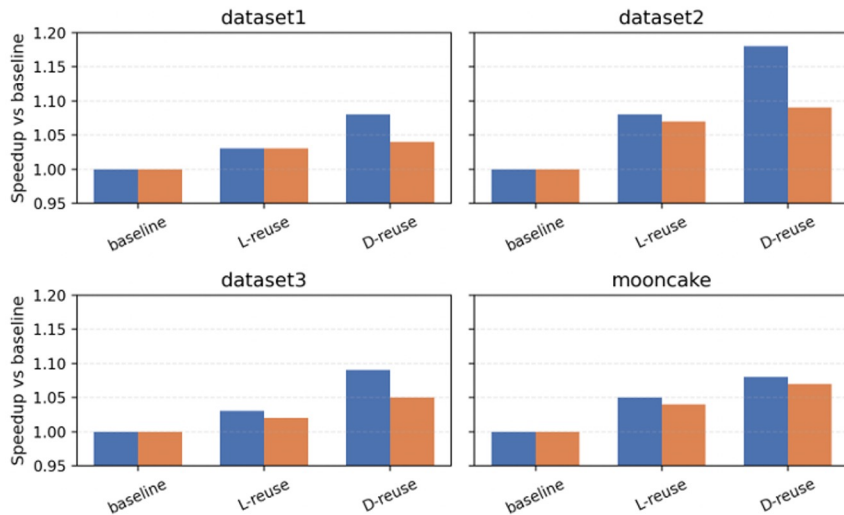
User case and Production Deployment

Distributed kvcache reuse perf

- Theoretical performance (speedup) under a specific hit rate is calculated by the actual recomputed tokens (compared with original number of tokens). It is a good metric to measure the overhead introduced by the process of kvcache reuse, such as data transfer, control logic and et.al.
- FlexKV has successful overhead control, local reuse introduces little overhead, and distributed reuse also has acceptable overhead.

FlexKV: theoretical vs observed TTFT speedup

theoretical
observed TTFT



Dynamo integration

```
export FLEXKV_CPU_CACHE_GB=32
export FLEXKV_SSD_CACHE_GB=128
export FLEXKV_SSD_CACHE_DIR="/workspace/dynamo-flexkv/flexkv_ssd1/"
export FLEXKV_SERVER_RECV_PORT="ipc:///tmp/flexkv_server_1"
CUDA_VISIBLE_DEVICES=0,1 \
DYN_VLLM_KV_EVENT_PORT=20080 \
VLLM_NIXL_SIDE_CHANNEL_PORT=20090 \
python -m dynamo.vllm \
    --model $MODEL_PATH \
    --served-model-name $MODEL_NAME \
    --tensor-parallel-size 2 \
    --connector "flexkv" \
    --enforce-eager \
    --gpu-memory-utilization 0.9 &
```

User guide and community



User [guide](#) for vllm

```
VLLM_USE_V1=1 python -m vllm.entrypoints.cli.main serve Qwen3/Qwen3-32B \
  --tensor-parallel-size 8 \
  --trust-remote-code \
  --port 30001 \
  --max-num-seqs 128 \
  --max-num-batched-tokens 8192 \
  --max_model_len 8192 \
  --max-seq-len-to-capture 8192 \
  --gpu-memory-utilization 0.8 \
  --enable-chunked-prefill \
  --enable-prefix-caching \
  --kv-transfer-config \
  '{"kv_connector": "FlexKVConnectorV1", "kv_role": "kv_both"}'
```

User guide for [trtllm](#)

User guide for Sglang

Integration for [Dynamo](#)

Contribute code:

- [Getting started](#)
- <https://github.com/taco-project/FlexKV>



LMCache and Advance KV cache techniques

Junchen Jiang
March 18th 2026



The Rise of **KV Cache**

Industry

More **KV caches** are
stored/ transferred

Academia

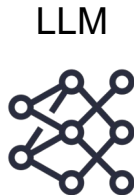
More **KV cache**-based
optimizations



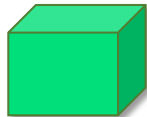
LM Cache
New Infra ??

KV Cache: A game changer

(Long)
Contexts



KV cache



KV cache is...

The intermediate
compute states

LLM's internal
understanding

**What LLM uses to
generate output**

So KV cache can be...

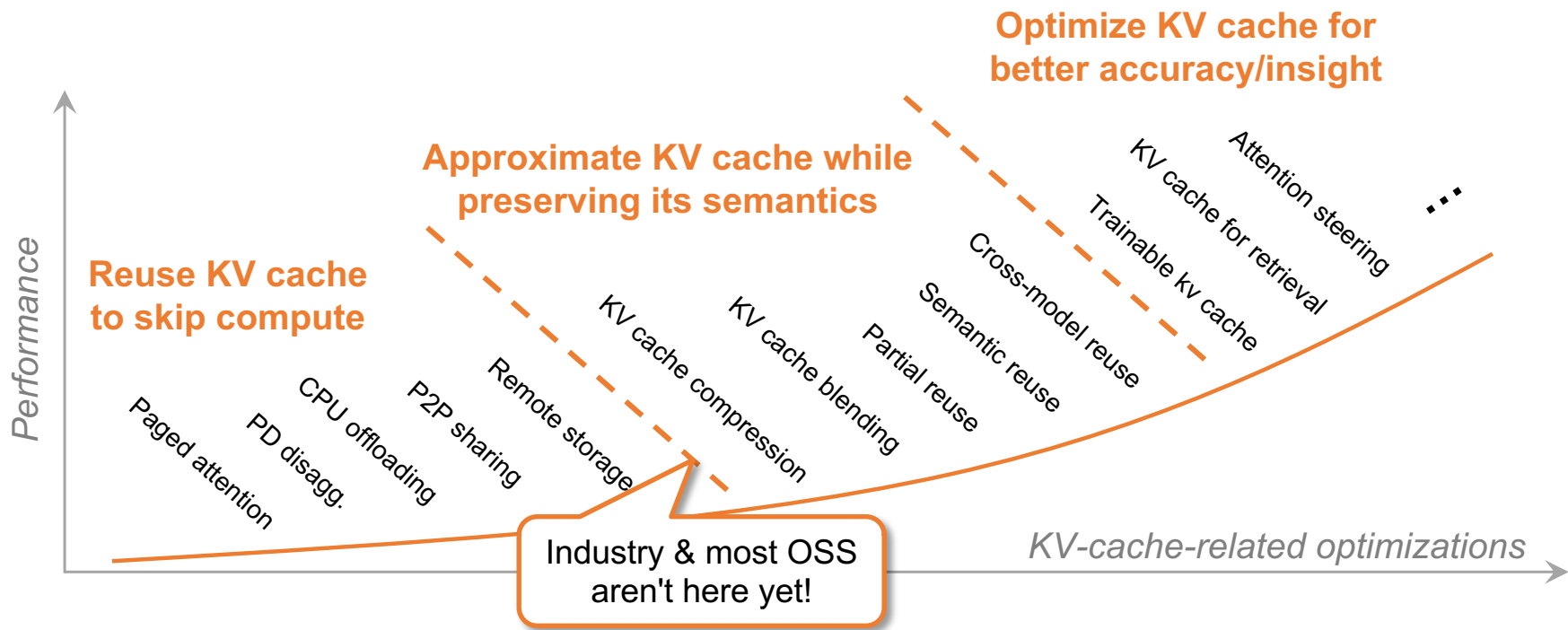
Reused to skip compute
and reduce delay

Approximated while
preserving its semantics

**Manipulated to influence
output accuracy**

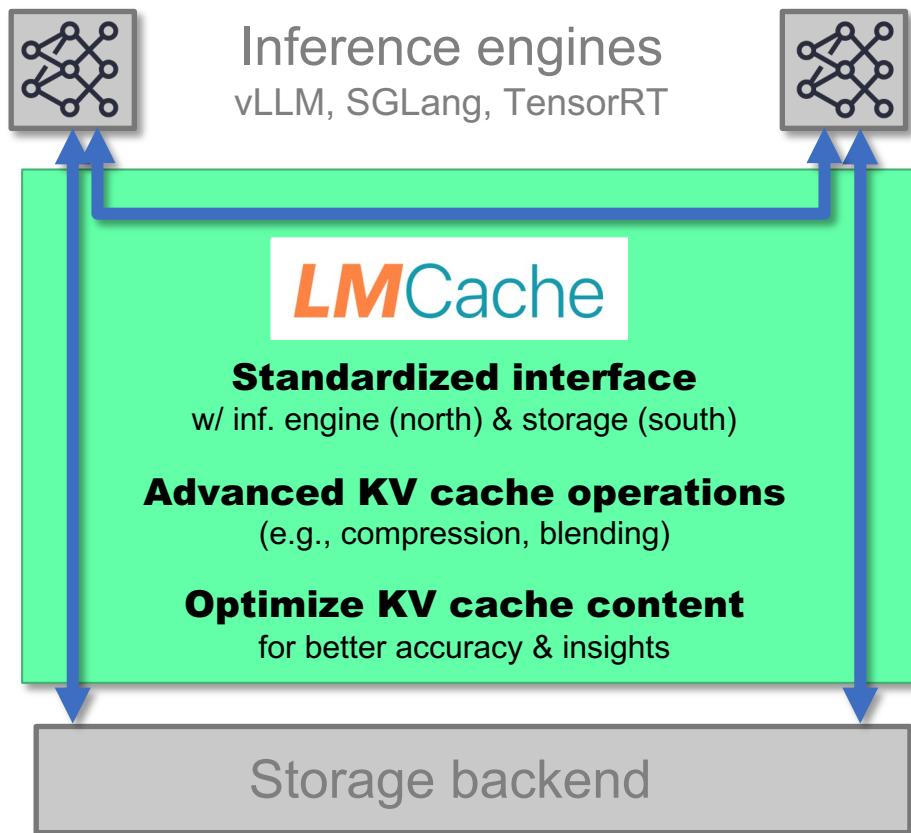
KV cache is not mere tensors,
it's the **AI-native data for the future!**

Many ideas, yet to be productized



Need a **New Infra** to **manage & optimize** KV cache!

LMCache + Tensormesh: 1st Infra for KV cache techniques



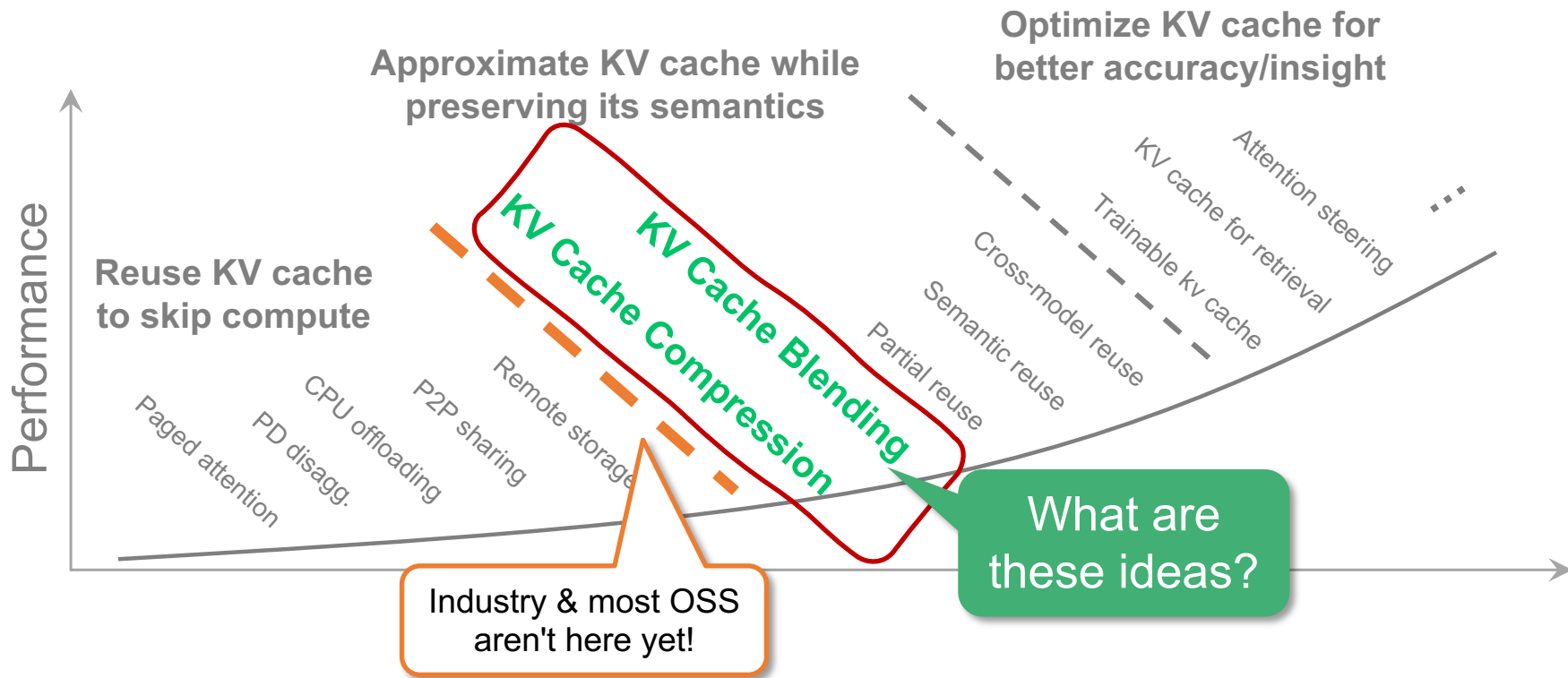
Design Choice

Decoupled from inference engines and storage



Faster innovation and development

Examples of advanced KV cache operation



What is

KV Cache Compression?

Why compression?

How much KV cache is generated by **one** B300 GPU **per day**?

= # of sec / day x tokens / sec x KV / token

86,400

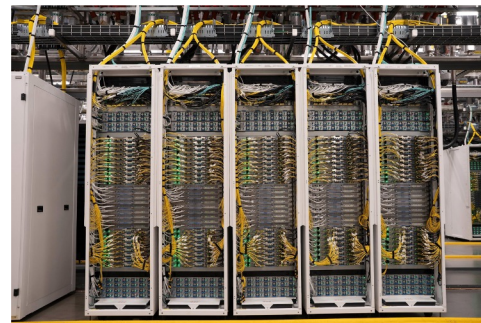
15,300

~70 KB

1M tok/s NVIDIA
GB300 NVL72

GPT-OSS-120b,
DeepSeek R1

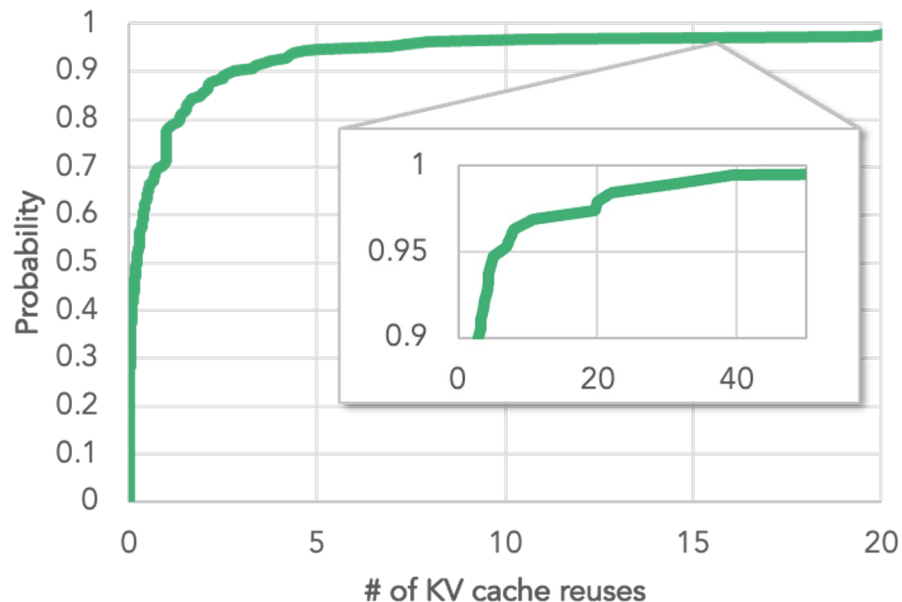
= **92.5 TB**



Courtesy Azure

GPUs **generate** lots of KV cache

Why storing KV cache?



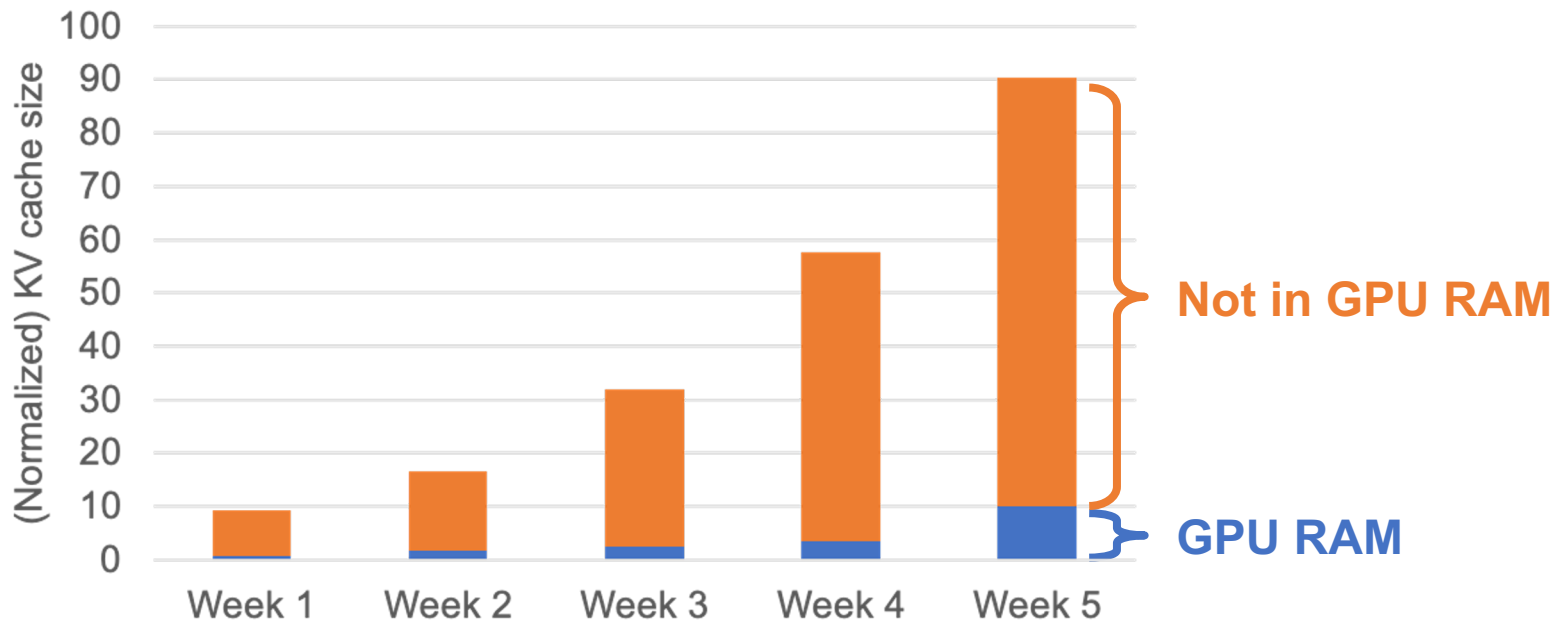
10% KV cache are reused **2.8+** times

1% KV cache are reused **30+** times

Storing more KV caches $\Rightarrow$
Saving more compute

People also **store** lots of KV cache

Where is KV cache stored?



Most KV cache is **not** in GPU RAM

Lots of KV cache are **generated** and need to be **stored**, ...

and most KV cache is **not** in GPU RAM



How to fit more KV
cache in limited space?

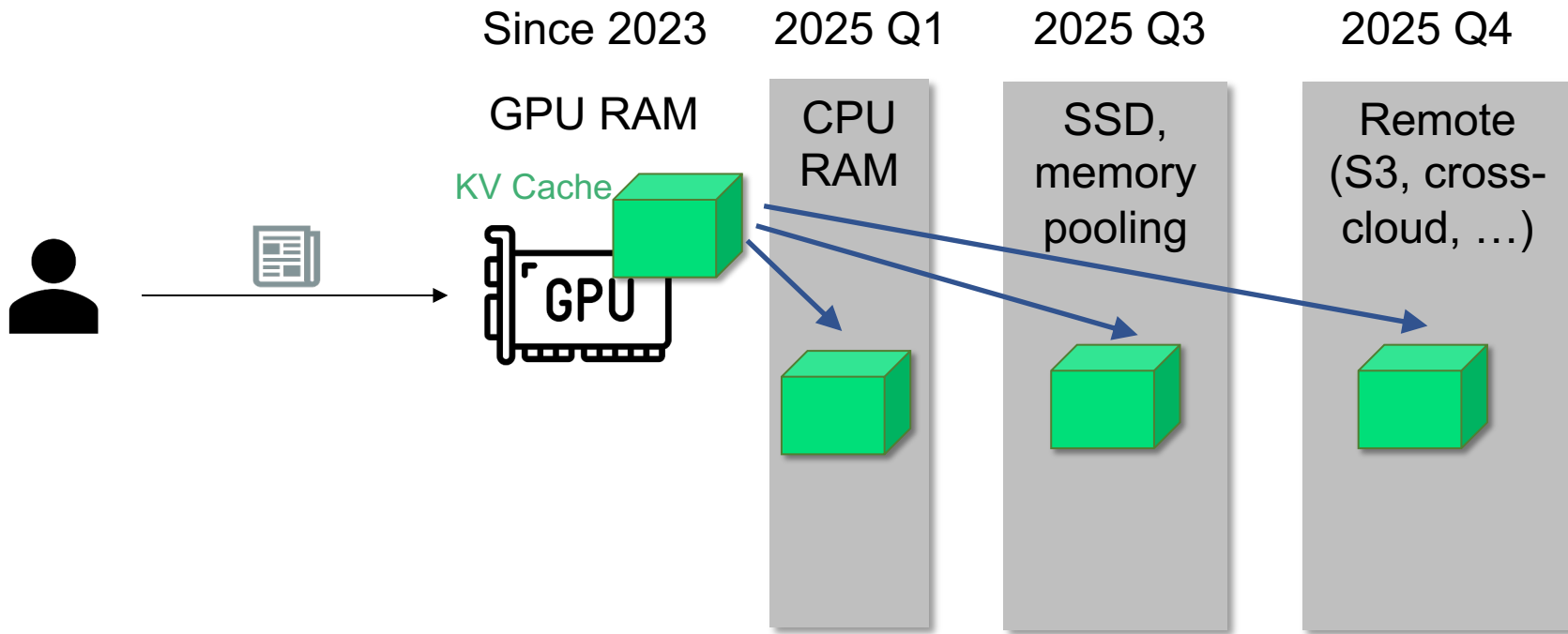


How to quickly load KV
cache back to GPU?



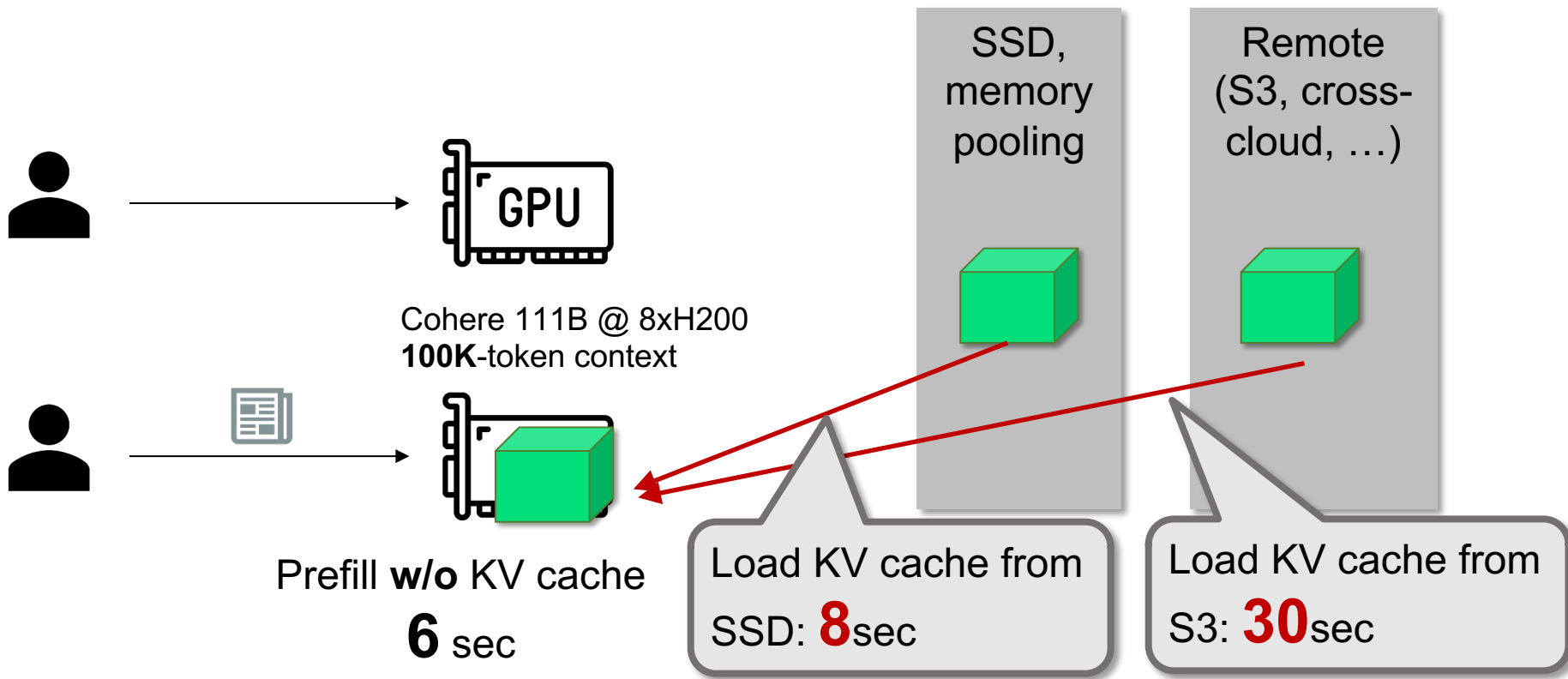
KV Cache **Compression**

Challenge of storing KV cache

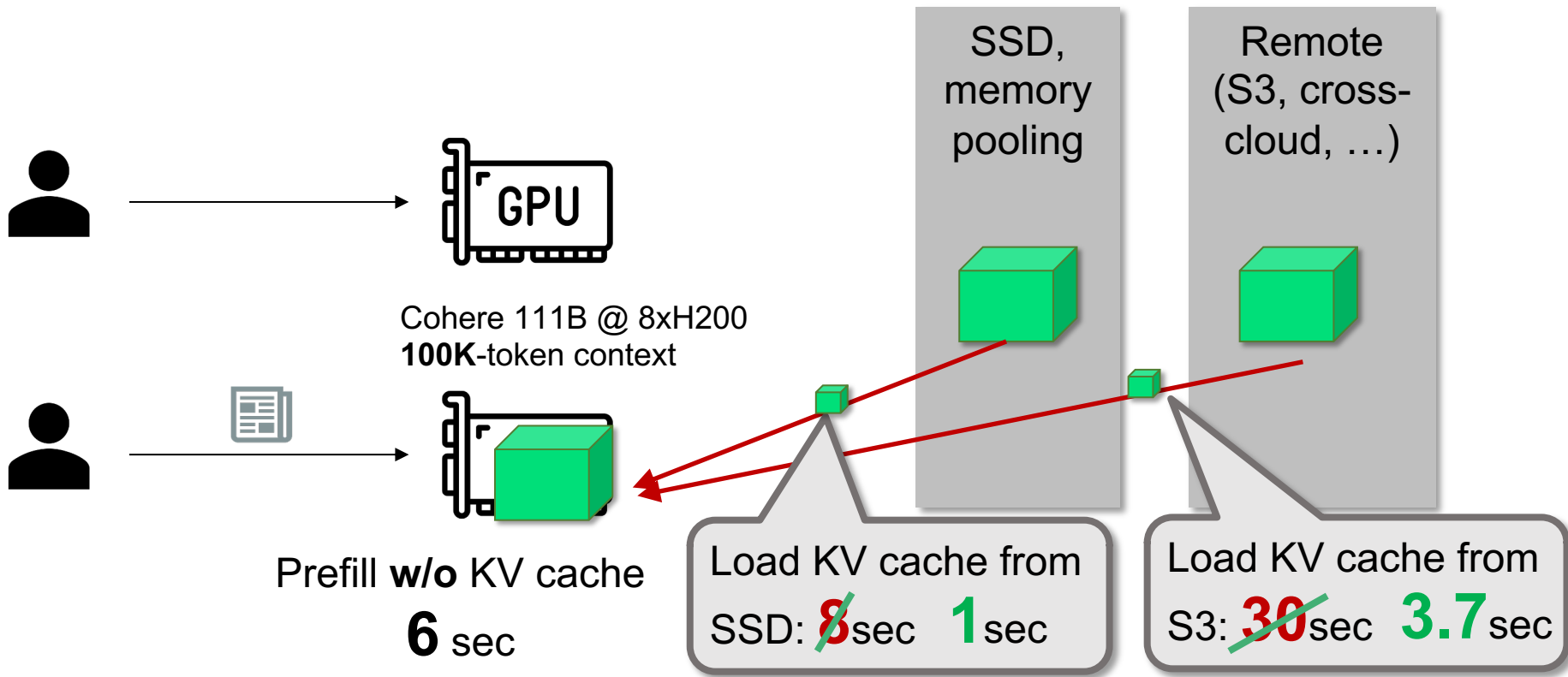


How to quickly load KV cache to GPU?

Loading KV cache from storage is SLOW!



Compression is key to faster KV cache movement



Design space of KV cache compression

Where to compress:

- **Inline-over network**
- In-GPU memory

When to compress:

- **Pre-query**
- Mid-query

Changes on KV cache:

- **Quantization**
- Token-level dropping
- Layer/head-wise dropping

An example: CacheGen [ACM SIGCOMM'24]

Available in **LMCache**

CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving

Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang*, Kuntai Du, Jiayi Yao, Shan Lu†, Ganesh Ananthanarayanan†, Michael Maire, Henry Hoffmann, Ari Holtzman, Junchen Jiang
 University of Chicago †Microsoft *Stanford University

Abstract

As large language models (LLMs) take on complex tasks, their inputs are supplemented with *longer contexts* that incorporate domain knowledge. Yet using long contexts is challenging as nothing can be generated until the whole context is processed by the LLM. While the context-processing delay can be reduced by reusing the KV cache of a context across different inputs, fetching the KV cache, which contains large tensors, over the network can cause high extra network delays.

CacheGen is a fast context-loading module for LLM systems. First, CacheGen uses a custom tensor encoder, leveraging KV cache's distributional properties to *encode* a KV cache into more compact

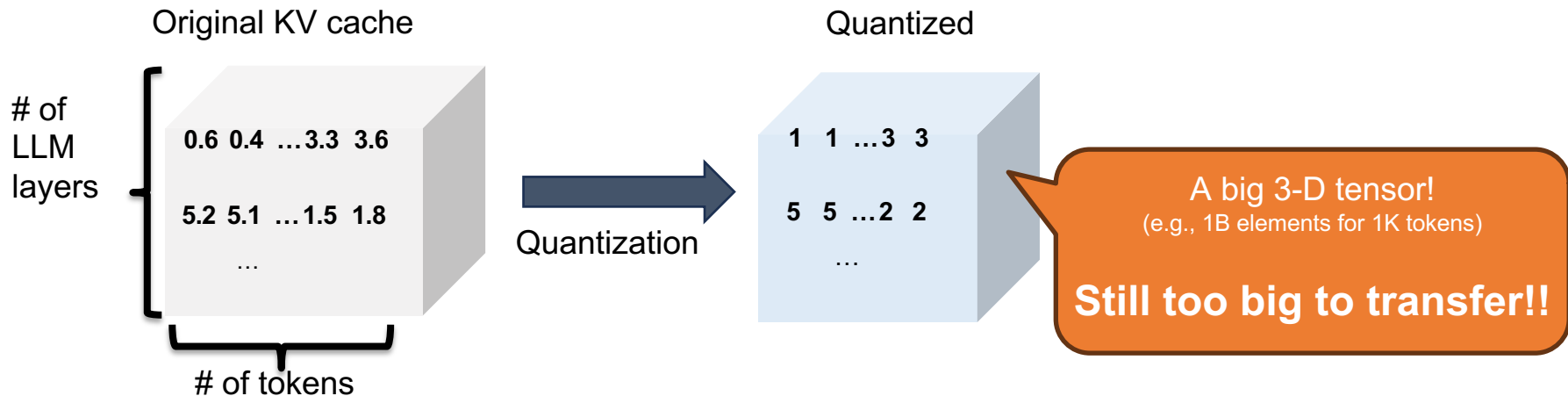
1 Introduction

With impressive generative quality, large language models (LLMs) are ubiquitously used [22, 38, 46, 128] in personal assistance, AI healthcare, and marketing. The wide use of LLM APIs (e.g., OpenAI GPT-4 [108]) and the industry-quality open-source models (e.g., Llama [129]), combined with popular application frameworks (e.g., HuggingFace [10], Langchain [83]), further boosts LLMs' popularity.

To perform complex tasks, users or applications often prepend an LLM input with a *long context* containing thousands of tokens or more. For example, some context supplements user prompts with domain-knowledge text so that the LLM can generate re-



CacheGen: an in-line compression example



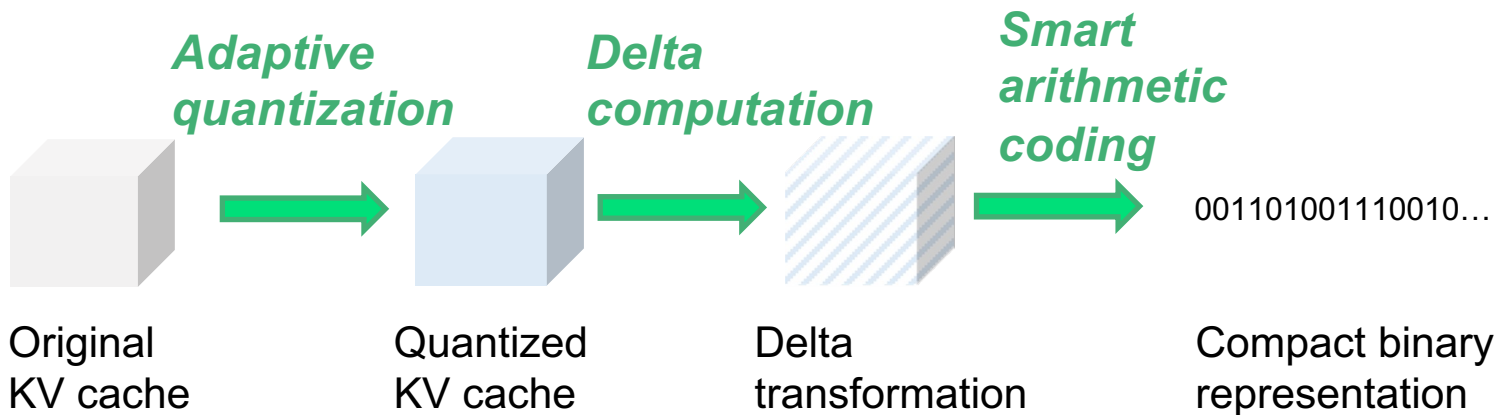
Insight: ***No longer*** need to keep KV cache's ***tensor shape*** if the goal is to reduce its size on ***network link or storage***

CacheGen: Encoding (quantized) KV cache like encoding videos

KV cache compression workflow

Algorithm insight:

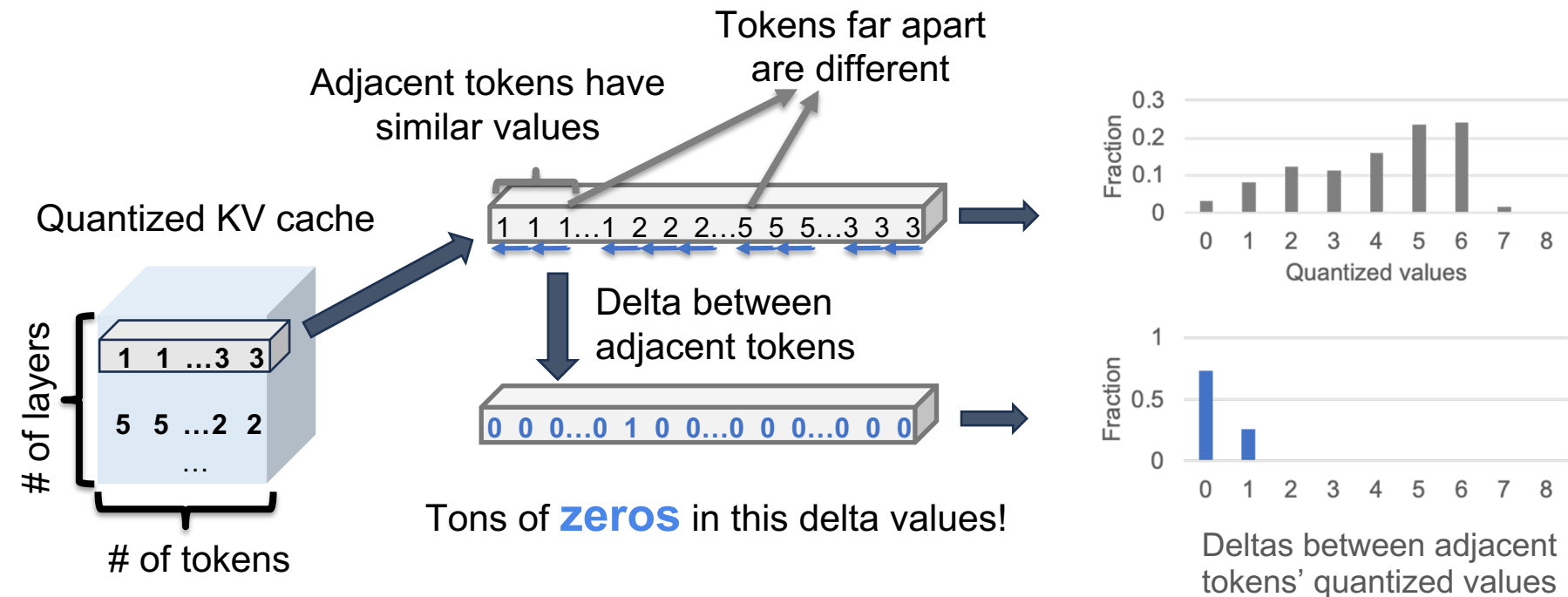
Neighboring tokens have similar values $\Rightarrow$ Compress their **deltas** instead



System insight:

Parallelizable by GPU $\Rightarrow$ little overhead compared to LLM inference

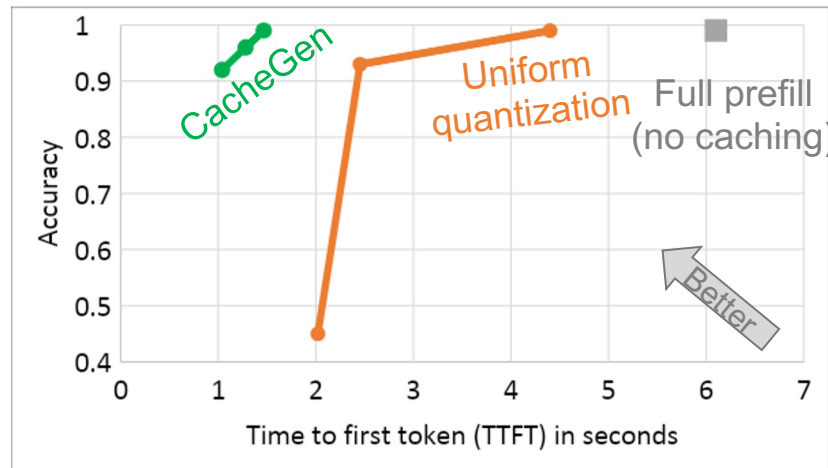
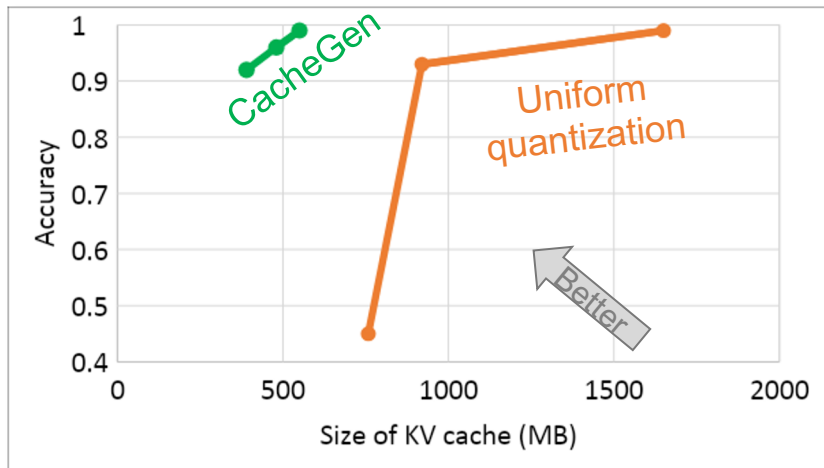
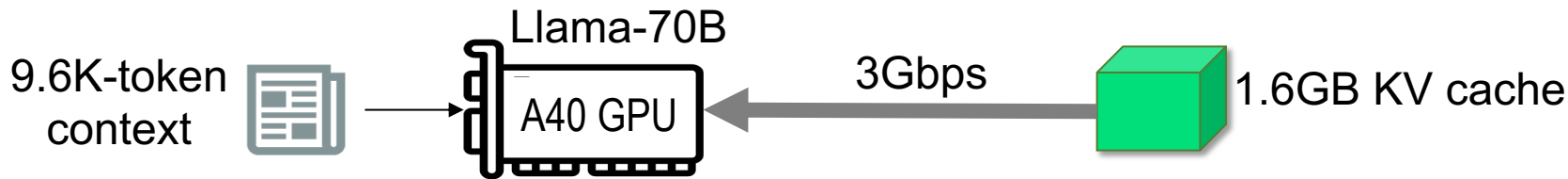
Leveraging statistic properties of KV cache



Courtesy [Yuhan Liu](#)

Delta values are much sparser $\Rightarrow$ Much more compact encodings

Effect of KV cache compression

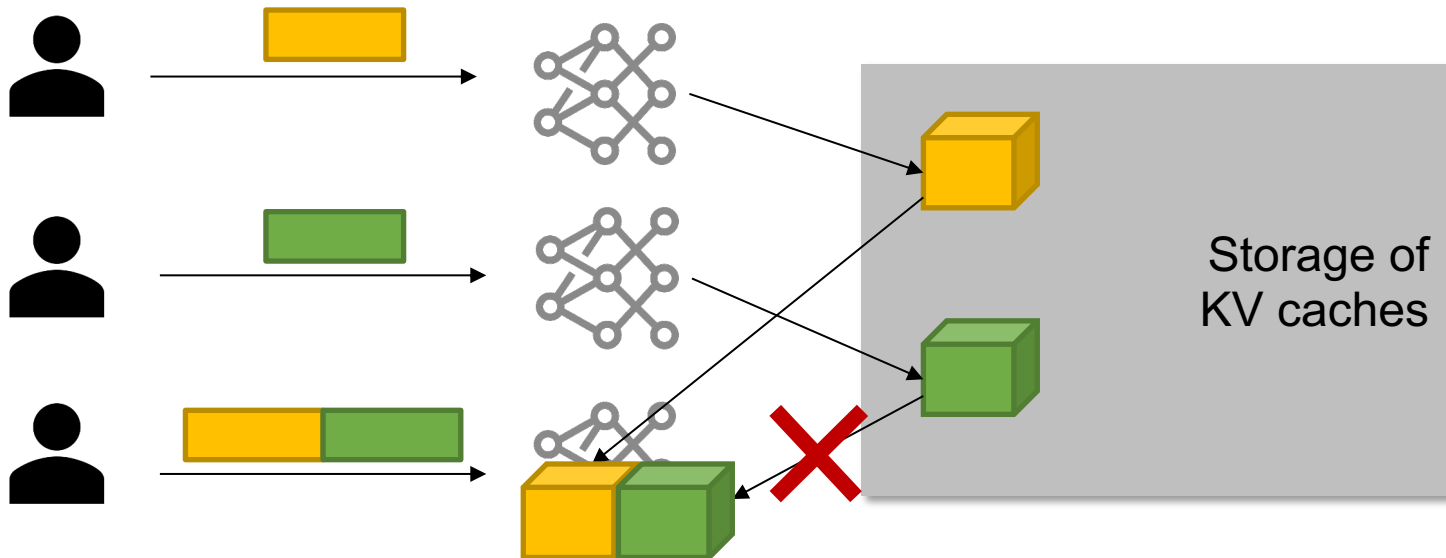


3x smaller than 8-bit quantization

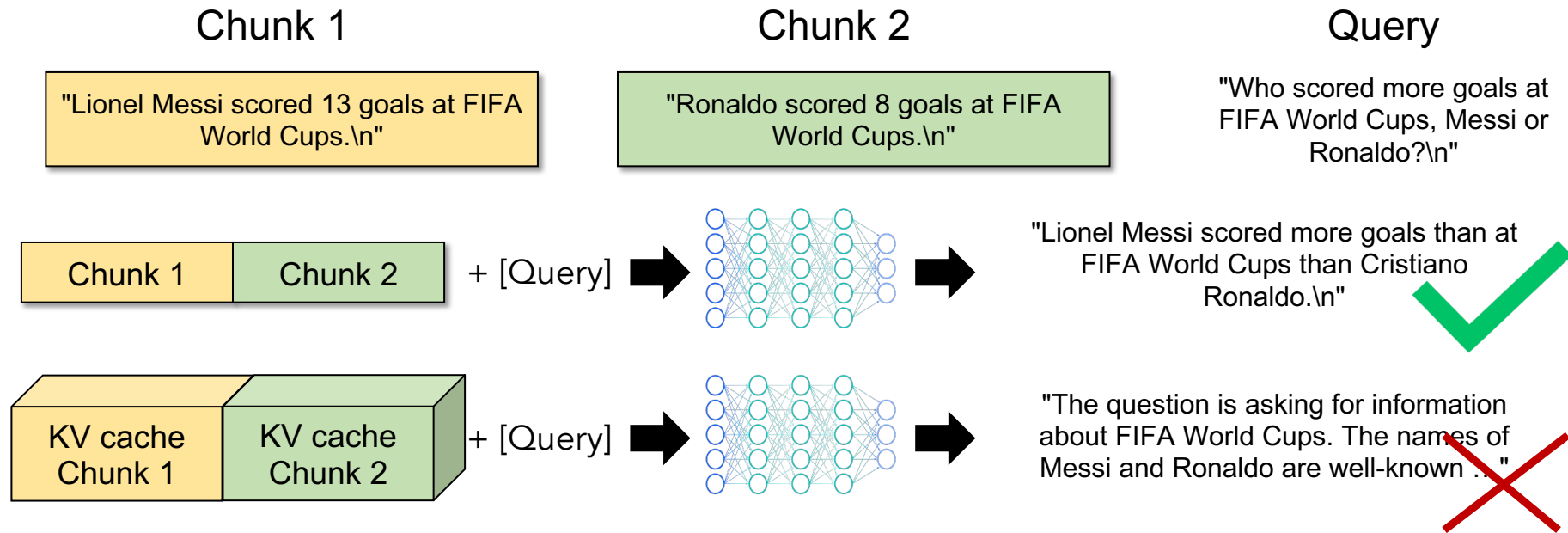
Much shorter time to first token

What is **KV Cache Blending?**

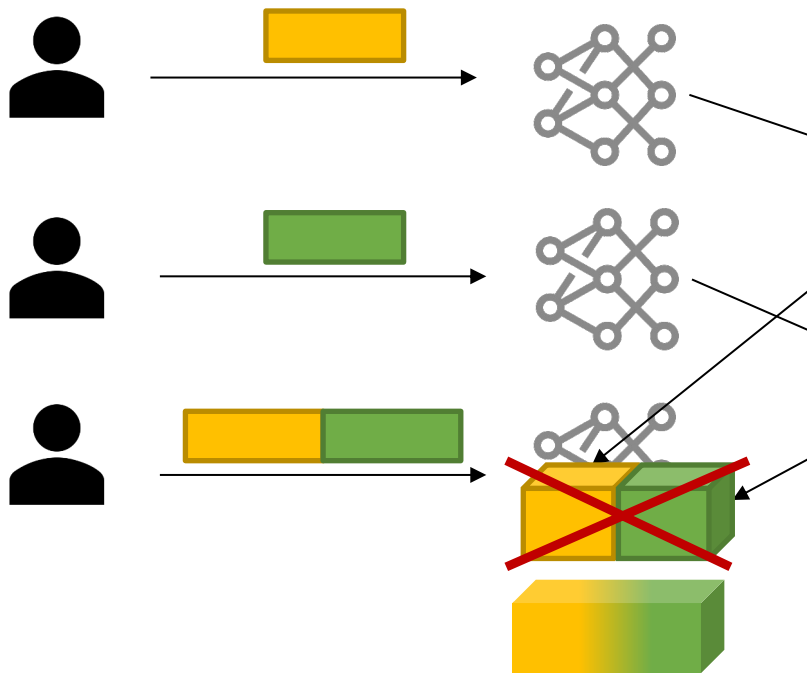
Prefix caching not sufficient



Non-prefix KV cache is not directly reusable



CacheBlend Illustration



CACHEBLEND: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion

Jiayi Yao
University of Chicago/CUHK Shenzhen

Siddhant Ray
University of Chicago

Kuntai Du
University of Chicago

Hanchen Li
University of Chicago

Yihua Cheng
University of Chicago

Shan Lu
Microsoft Research / University of Chicago

Yuhan Liu
University of Chicago

Qizheng Zhang
Stanford University

Junchen Jiang
University of Chicago

Abstract

Large language models (LLMs) often incorporate multiple text chunks in their inputs to provide the necessary contexts. To speed up the prefill of the long LLM inputs, one can *pre-compute* the KV cache of a text and *re-use* the KV cache when the context is reused as the prefix of another LLM input. However, the reused text chunks are *not* always the input prefix, which makes precomputed KV caches not directly usable since they ignore the text's *cross-attention* with the preceding texts. Thus, the benefits of reusing KV caches remain largely unrealized.

This paper tackles just one challenge: when an LLM input contains *multiple* text chunks, *how to quickly combine their precomputed KV caches* in order to achieve the same generation quality as the expensive full prefill (*i.e.*, without reusing KV cache)? This challenge naturally arises in retrieval-augmented generation (RAG) where the input is supplemented with multiple retrieved texts as the context. We present CACHEBLEND, a scheme that reuses the pre-computed KV caches, regardless prefix or not, and *selectively recomputes the KV values of a small subset of tokens*

with the state-of-the-art KV cache reusing schemes on three open-source LLMs of various sizes and four popular benchmark datasets of different tasks, we show that CACHEBLEND reduces time-to-first-token (TTFT) by 2.2–3.3x and increases the inference throughput by 2.8–5x from full KV recompute without compromising generation quality. The code is available at <https://github.com/LMCache/LMCache>.

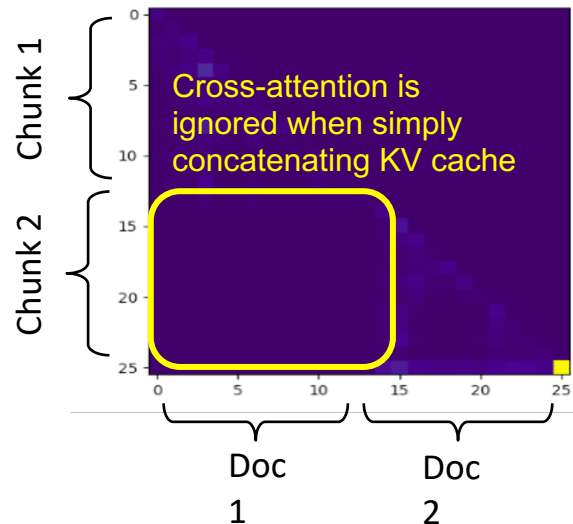
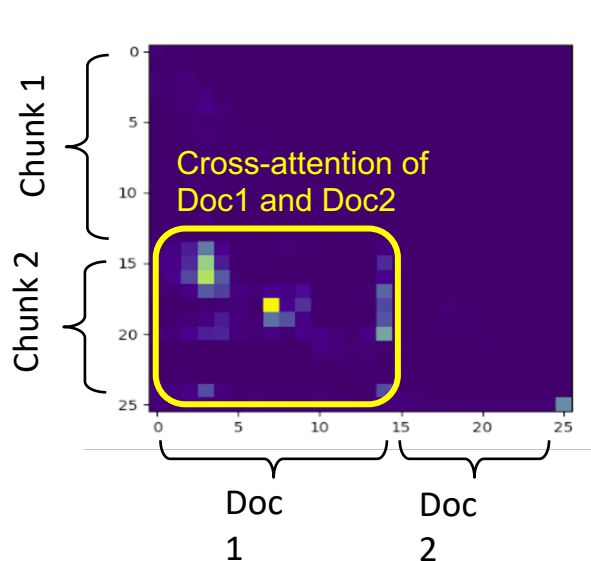
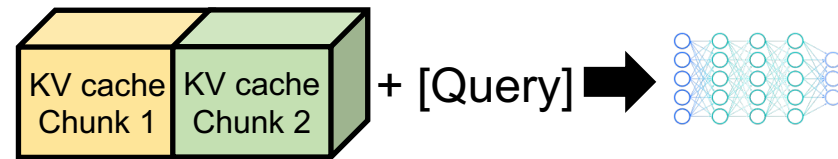
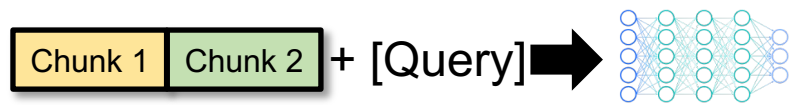
CCS Concepts • Computing methodologies → Natural language processing; • Networks → Cloud computing; • Information systems → Data management systems.

Keywords Large Language Models, KV Cache, Retrieval-Augmented-Generation

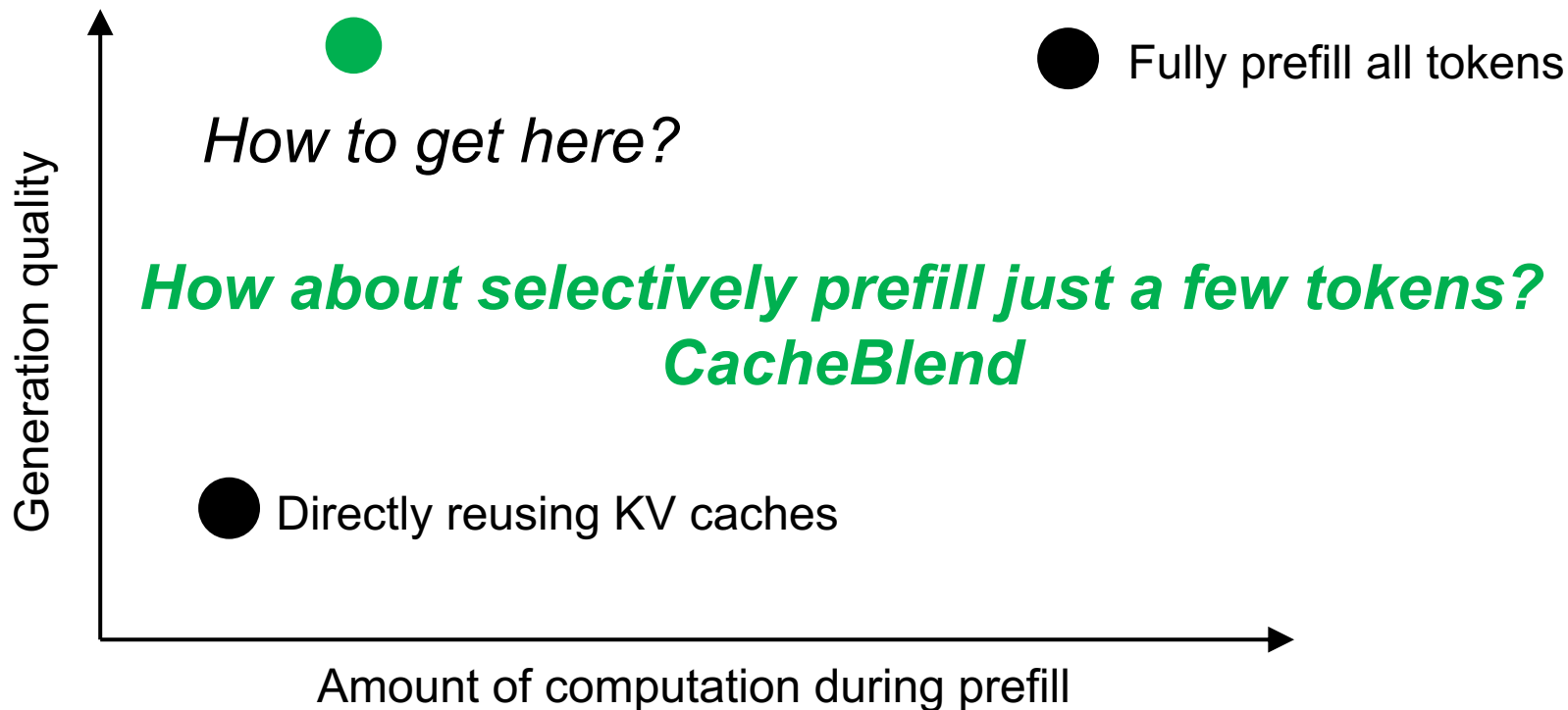
ACM Reference Format:

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CACHEBLEND: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Twentieth European Conference on Computer Systems (EuroSys '25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3689031.3696098>

Why? Missing cross-attention



Computation vs. quality



Full prefill vs. Selective prefill (CacheBlend)

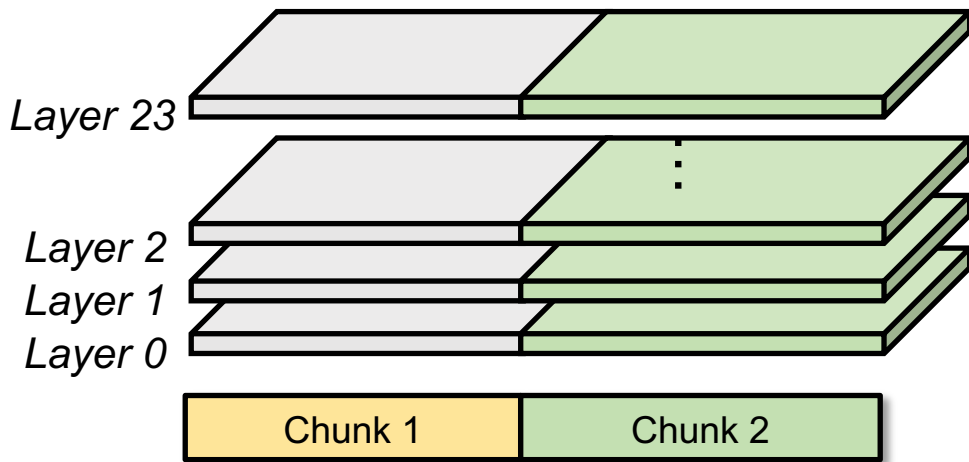


Reused cache



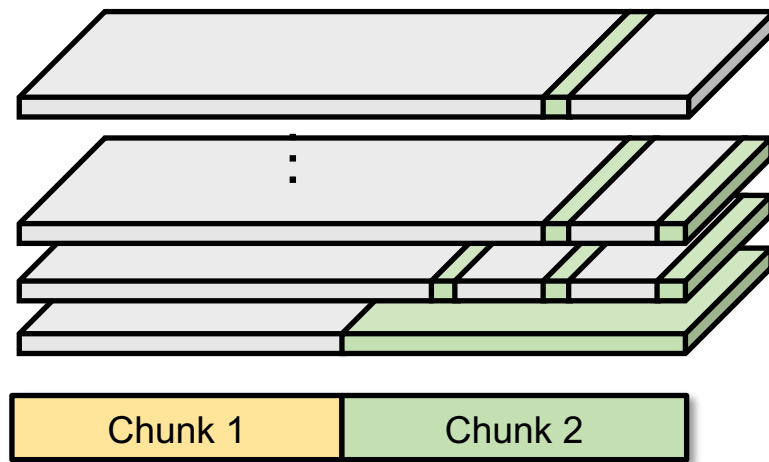
Recomputed cache

Full prefill



Compute the KV of all tokens
of Doc 2 on all layers

Selective prefill (CacheBlend)

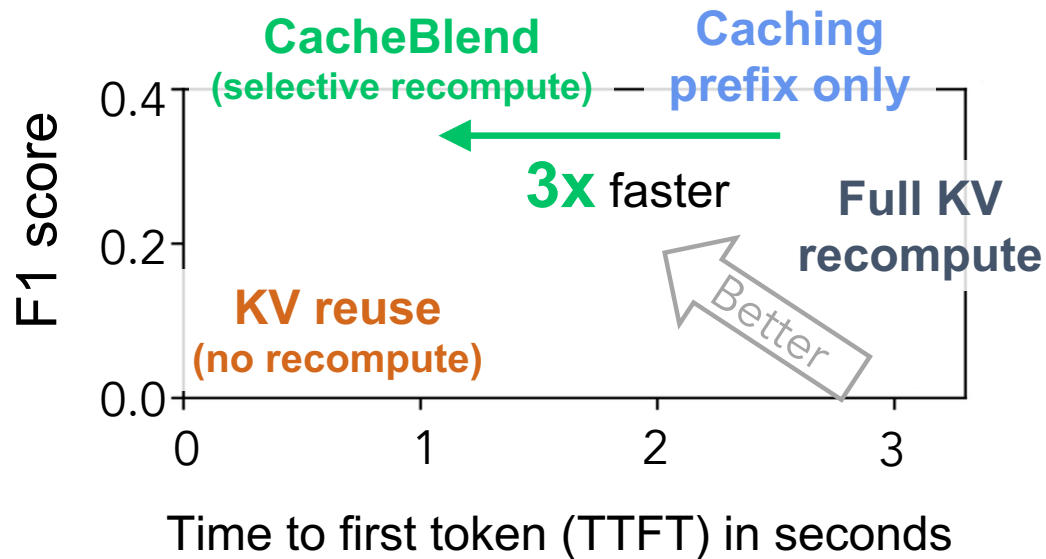


Only recompute the KV of a few
selected tokens on each layer

Effect of CacheBlend: Faster response

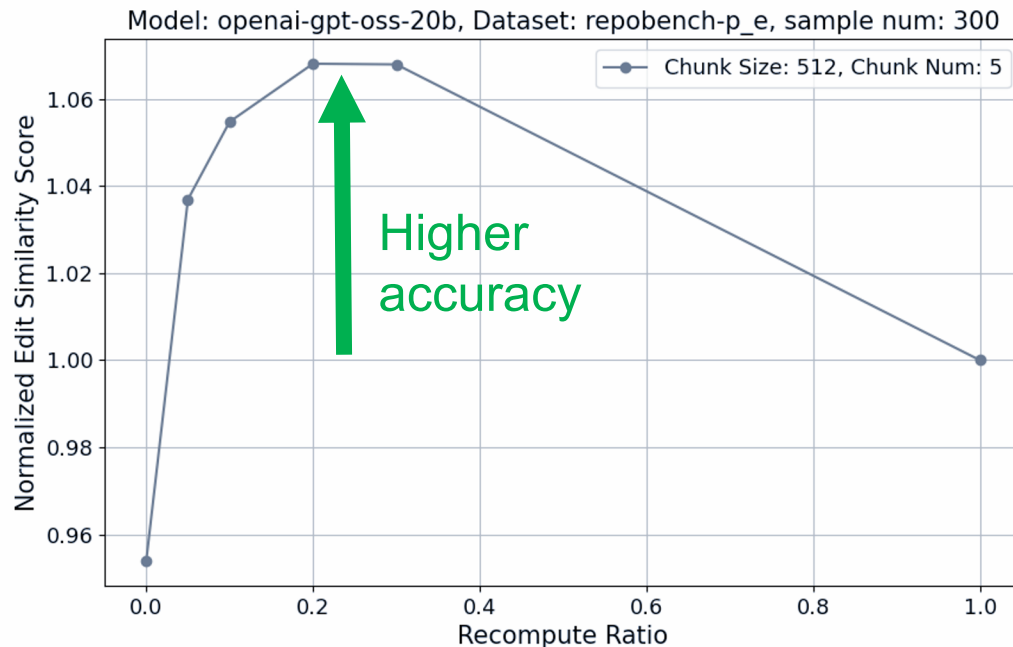
Setup:

- Dataset: "2WikiMQA"
 - Top 6 chunks (512 tokens each)
- 1.5K sampled queries
- Model: Llama 70B
- GPU: 2x A40
- KV cache initially stored on disk



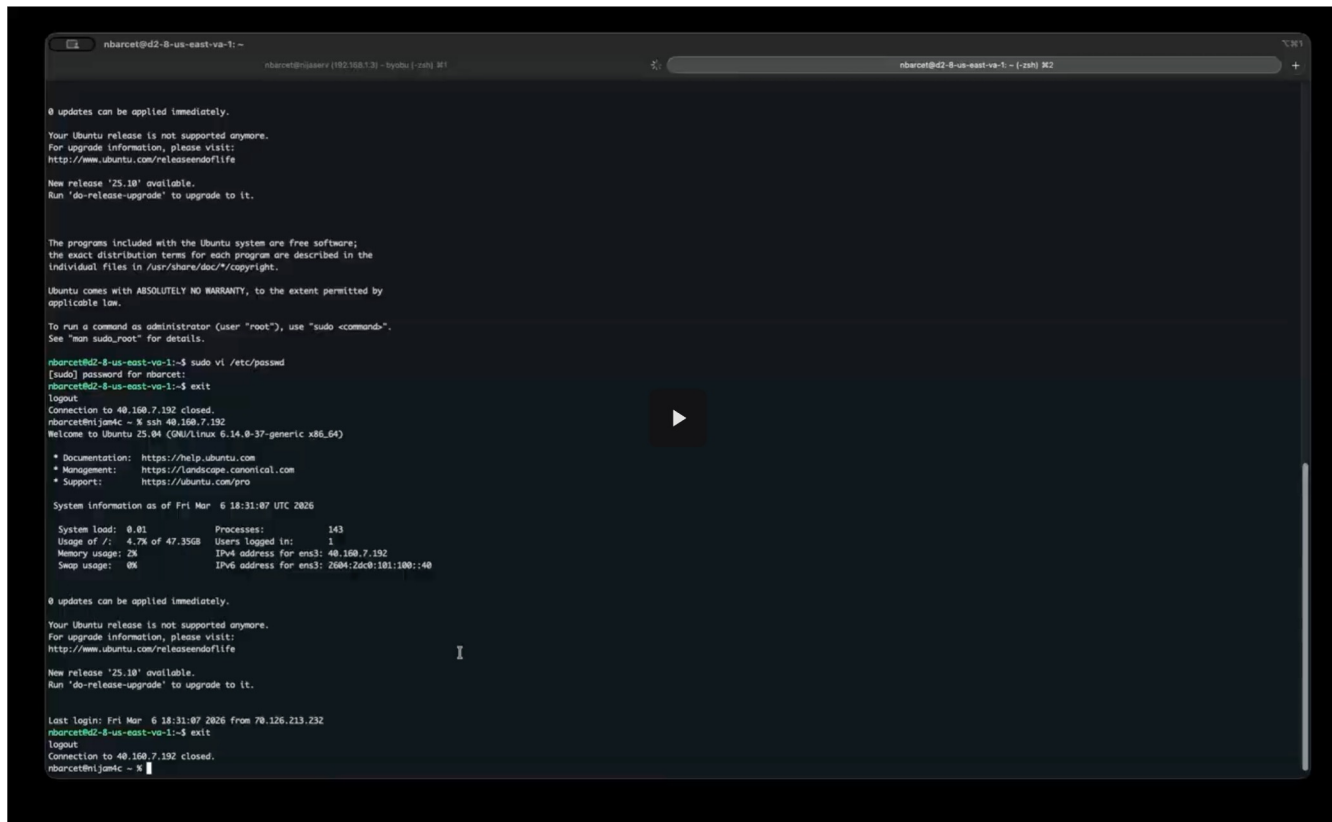
Effect of CacheBlend: Higher response accuracy

Model: gpt-oss-20b
Task: Code completion
Accuracy: Output edit similarity



Beyond speedup, CacheBlend can also improve inference accuracy.

Demo of OpenClaw improved by CacheBlend (via Tensormesh SaaS)



```

nbarcet@d2-8-us-east-v1-~
nbarcet@njasrv (192.168.1.3) - byobu (-zsh) X1

@ updates can be applied immediately.

Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releases/oldlife

New release '25.10' available.
Run 'do-release-upgrade' to upgrade to it.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

nbarcet@d2-8-us-east-v1-~$ sudo vi /etc/passwd
[sudo] password for nbarcet:
nbarcet@d2-8-us-east-v1-~$ exit
logout
Connection to 48.168.7.192 closed.
nbarcet@njasrv ~$ ssh 48.168.7.192
Welcome to Ubuntu 25.04 (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Mar 6 18:31:07 UTC 2026

System load: 0.01          Processes:            143
Usage of /:  4.7% of 47.3GB Users logged in:      1
Memory usage: 2%          IPv4 address for ens3: 48.168.7.192
Swap usage:  0%           IPv6 address for ens3: 2604:2dc0:101:100::40

@ updates can be applied immediately.

Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releases/oldlife

New release '25.10' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Mar 6 18:31:07 2026 from 70.126.213.232
nbarcet@d2-8-us-east-v1-~$ exit
logout
Connection to 48.168.7.192 closed.
nbarcet@njasrv ~$

```

Integrated in
NVIDIA Reachy
Demo

More demos at our
booth #7022

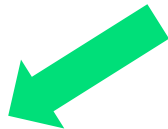
KV cache techniques
from academia

More KV cache stored
by the industry

**Your next
idea!**



LMCache

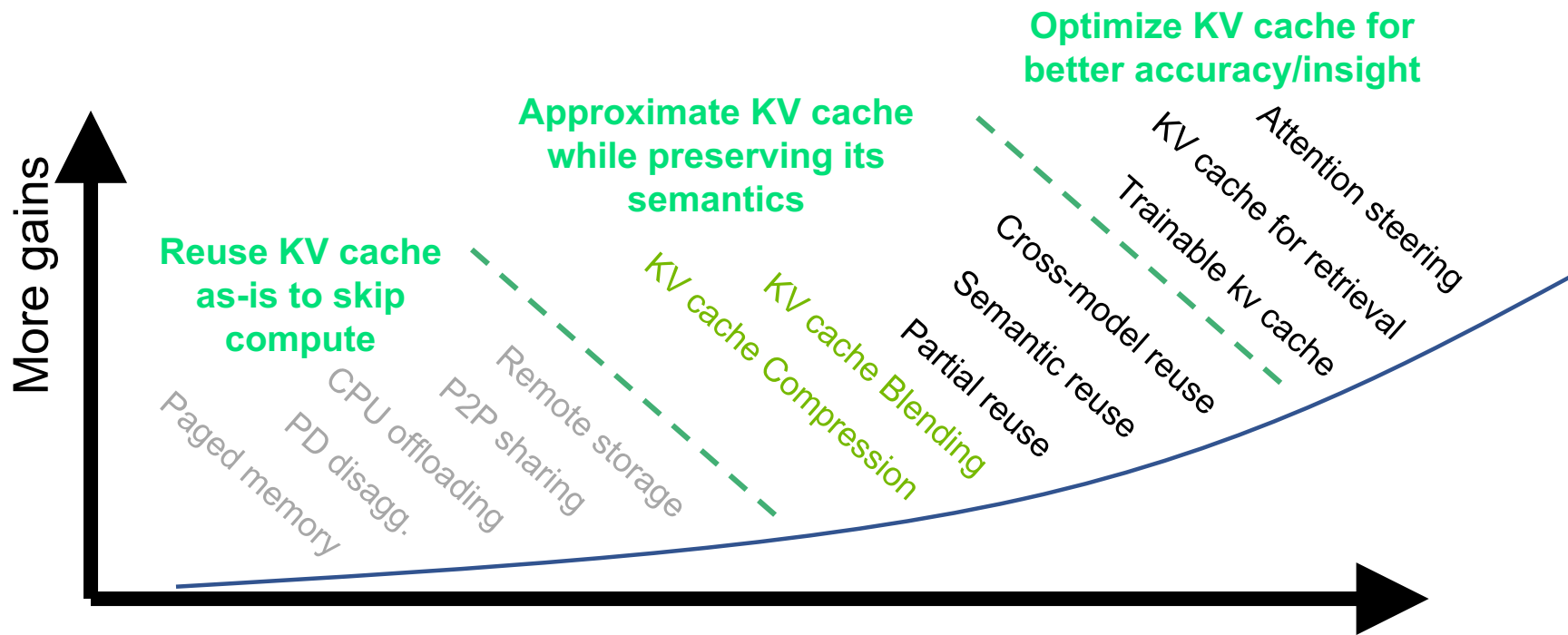


Inference engines
vLLM, SGLang, TensorRT

 *tensormesh*
Inference SaaS

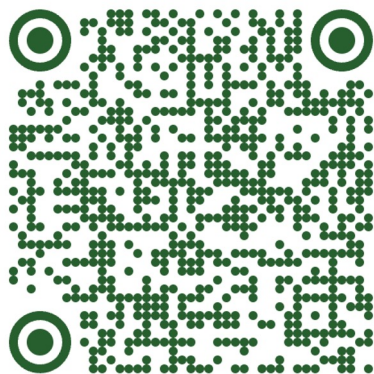
Storage partners

Conclusion



Exciting future awaits

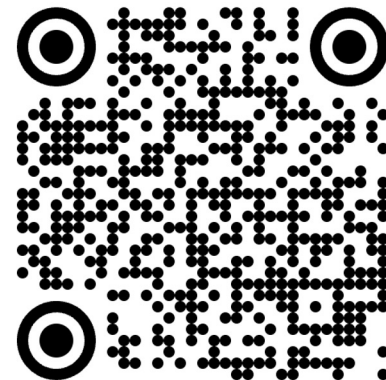
Conclusion



`pip install kvbm`



`https://github.com/taco-project/FlexKV`



`pip install lmcache`

